A Project Report On

BRAIN STROKE DETECTION USING RESNET-50

Submitted in partial fulfillment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

in

COMPUTER SCIENCE & ENGINEERING

(ARTIFICIAL INTELLIGENCE & MACHINE LEARNING)

from

JAWAHARLAL NEHRU TECHNOLOGICAL UNIVERSITY HYDERABAD

By

BELLAMKONDA BHARATH (22C11A6607)

BIJIGA VINAY (22C11A6643)

Under the guidance of

Mrs. N. MOWNIKA **M.Tech., (Ph.D).**

Assistant Professor

ANURAG ENGINEERING COLLEGE

(An Autonomous Institution)

(Affiliated to JNTUH, Hyderabad & Approved by AICTE, New Delhi)

Ananthagiri (V & M), Kodad, Suryapeta (Dt), Telangana -508206.

2025-26

ANURAG ENGINEERING COLLEGE

(An Autonomous Institution)

(Affiliated to JNTUH, Hyderabad & Approved by AICTE, New Delhi)
Ananthagiri (V & M), Kodad, Suryapet (Dt), Telangana - 508206.

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING (ARTIFICIAL INTELLIGENCE & MACHINE LEARNING)

CERTIFICATE

*This is to certify that the project work entitled "**BRAIN STROKE DETECTION USING RESNET-50**" is a bonafide work done by "**BELLAMKONDA BHARATH (22C11A6607) and BIJIGA VINAY (22C11A6643),**" in the partial fulfillment for the award of Bachelor of Technology in CSE (Artificial Intelligence & Machine Learning) from JNTUH, Hyderabad during the year **2025-26**.*

This work has not been submitted to any other university or institute or organization for the award of any degree or diploma.

Mrs. N .Mownika
SUPERVISOR

Mrs. T. Indira Priyadarshini
HoD – CSE(AI&ML)

EXTERNAL EXAMINER

ACKNOWLEDGEMENTS

We would like to express our sincere gratitude to all those who have contributed, directly or indirectly, to the successful completion of this project.

We are highly indebted to our project supervisor, **Mrs. N. Mownika**, for her continuous guidance, valuable suggestions, and constant encouragement throughout the course of this project.

We express our heartfelt thanks to **Mrs. T. Indira Priyadarshini**, HoD, Department of CSE (AI & ML), for her support and motivation during the development of this work.

We also extend our sincere thanks to all the **teaching and non-teaching staff members of the Department of CSE (AI & ML)** for their cooperation and assistance.

We take this opportunity to express our gratitude to the Principal, **Dr. T. Suresh Kumar**, for providing the necessary academic environment and encouragement.

We are thankful to the **Management of Anurag Engineering College** for providing the infrastructure and facilities required to carry out this project.

Finally, we would like to thank our **parents and friends** for their constant support, encouragement, and motivation.

BELLAMKONDA BHARATH
(22C11A6607)

BIJIGA VINAY
(22C11A6643)

BRAIN STROKE DETECTION USING RESNET-50

CONTENTS

Abstract			i
List Of Figures			ii
List Of Tables			iii
List Of Acronyms			iv
1	Introduction		1
	1.1	Background	2
	1.2	Problem statement	4
	1.3	Objectives	5
2	System Analysis		7
	2.1	Literature survey	7
	2.2	Dataset Description	11
	2.3	System Specifications	13
	2.4	Technical feasibility	14
3	System Design		15
	3.1	Architecture	15
	3.2	Module Design	18
	3.3	UML Diagrams	20
4	Implementation		24
	4.1	Technologies Used	24
	4.2	Algorithm	25
	4.3	Training Process	28
	4.4	User Interface Screens	31
	4.5	Sample Code	38
5	Testing and Evaluation		42
	5.1	Testing Methodology	42
	5.2	Evaluation Metrics	44
	5.3	Test Cases	46
6	Results and Discussion		49
	6.1	Performance Analysis	49
	6.2	Graphical Analysis	51
7	Conclusion and Future Scope		57
	Bibliography		59

Abstract

Brain stroke is a severe neurological disorder that occurs when the blood supply to a part of the brain is disrupted, leading to rapid damage of brain tissues and potentially resulting in permanent disability or death. Early and accurate detection of stroke is therefore essential for timely medical intervention and improved patient outcomes. This project presents an automated brain stroke detection system based on deep learning techniques for the analysis of Computed Tomography (CT) brain images. The proposed system employs the ResNet-50 architecture as the primary deep feature extraction backbone to learn hierarchical representations from CT scans. To further enhance the detection of subtle spatial patterns associated with stroke, a SqueezeNet-inspired Fire module is integrated after the ResNet feature extractor to refine the learned feature maps. The dataset used in this study consists of labeled CT brain images categorized into normal and stroke classes. Prior to training, preprocessing operations including image resizing, normalization, and data augmentation are applied to improve model robustness and generalization. The model is trained using a supervised learning approach and evaluated using standard performance metrics such as accuracy, precision, recall, F1-score, and ROC-AUC. Experimental results demonstrate that the proposed architecture effectively identifies stroke-related abnormalities in CT images and achieves reliable classification performance. The developed system also supports visualization techniques such as Grad-CAM to highlight the regions influencing model predictions, thereby improving interpretability. Overall, the proposed deep learning framework demonstrates the potential of artificial intelligence–based systems in assisting medical image analysis and supporting healthcare professionals in the early detection of brain stroke.

Keywords: Brain Stroke Detection, Deep Learning, ResNet-50 Architecture, CT Brain Images, Feature Extraction, Grad-CAM Visualization

.

List Of Figures

Figure No.	Description	Page No.
3.1	System Architecture	15
3.2	ResNet Architecture	16
3.3	SqueezeNet Architecture	17
3.4	Usecase Diagram	20
3.5	Class Diagram	21
3.6	Sequence Diagram	22
3.7	Activity Diagram	23
4.1	Login Page	31
4.2	Home Page	32
4.3	CT Scan Image Selection Interface	33
4.4	Stroke Prediction Result Interface	34
4.5	About Page	35
4.6	Contact Page	36
4.7	Admin Dashboard Page	37
5.1	Classification Report	44
6.1	Training and Validation accuracy graph	51
6.2	Training and Validation loss graph	52
6.3	Confusion Matrix visualization	53
6.4	ROC Curve	54
6.5	Performance Comparison Graph with AUC	55

List Of Tables

Table No.	Description	Page No.
2.1	Literature Survey	9
2.2	Hardware Requirements	13
2.3	Software Requirements	13
5.1	Test Cases	47
6.1	Performance Comparison of Different Models	55

List Of Acronyms

AI	Artificial Intelligence
API	Application Programming Interface
AUC	Area Under Curve
CNN	Convolutional Neural Network
CPU	Central Processing Unit
CSS	Cascading Style Sheets
CT	Computed Tomography
CVA	Cerebrovascular Accident
DL	Deep Learning
FPR	False Positive Rate
F1-Score	F1 Score (Harmonic Mean of Precision and Recall)
GB	Gigabytes
GLCM	Gray Level Co-occurrence Matrix
GPU	Graphics Processing Unit
HOG	Histogram of Oriented Gradients
HTML	HyperText Markup Language
JPG	Joint Photographic Experts Group
KNN	K-Nearest Neighbors
LBP	Local Binary Pattern
ML	Machine Learning
MRI	Magnetic Resonance Imaging
MySQL	My Structured Query Language
NumPy	Numerical Python
PNG	Portable Network Graphics
RAM	Random Access Memory
ReLU	Rectified Linear Unit
ResNet	Residual Network
ROC	Receiver Operating Characteristic
SQLite	Structured Query Language Lite
SVM	Support Vector Machine
TPR	True Positive Rate
UI	User Interface
UML	Unified Modeling Language

CHAPTER 1

INTRODUCTION

Brain stroke is one of the leading causes of death and long-term disability worldwide, posing a significant challenge to modern healthcare systems. According to global health statistics, stroke accounts for a substantial percentage of mortality and neurological disorders, making early detection and timely diagnosis critically important. With the rapid advancement of medical imaging technologies and the increasing availability of healthcare data, artificial intelligence has emerged as a powerful tool for improving disease detection and clinical decision-making processes [1].

Traditional methods for diagnosing stroke primarily involve clinical evaluation and imaging techniques such as Computed Tomography (CT) scans and Magnetic Resonance Imaging (MRI). These methods are effective but often require expert interpretation, making the process time-consuming and dependent on radiologists' experience. In emergency cases, delays in diagnosis can significantly affect patient outcomes. To overcome these limitations, machine learning approaches have been introduced to assist in automated stroke detection. Techniques such as Support Vector Machines (SVM), Decision Trees, and Random Forests have been widely used to analyze medical images and patient data [2]. However, these methods rely heavily on manual feature extraction and may not effectively capture complex spatial patterns present in medical images.

In recent years, deep learning techniques, particularly Convolutional Neural Networks (CNNs), have gained significant attention due to their ability to automatically learn hierarchical features from image data. CNN-based models have demonstrated superior performance in medical image classification tasks, including stroke detection from CT scans [3]. These models eliminate the need for manual feature engineering and are capable of capturing intricate patterns and variations in brain images, thereby improving diagnostic accuracy [4]. Despite these advantages, deep learning models can suffer from issues such as overfitting, high computational requirements, and sensitivity to dataset size and quality.

To address these challenges, advanced architectures such as ResNet50 have been introduced, which utilize residual learning to enable deeper networks without degradation in performance. Additionally, hybrid approaches that integrate feature refinement techniques further enhance model efficiency and accuracy. By combining deep learning with optimized preprocessing and feature extraction methods, the proposed system aims to achieve reliable and efficient brain stroke detection. This approach not only improves prediction accuracy but also supports faster and more consistent clinical decision-making in healthcare environments.

1.1 Background

Brain stroke, also known as a cerebrovascular accident (CVA), is a critical neurological condition that occurs when the blood supply to a part of the brain is interrupted or significantly reduced. This disruption prevents brain tissue from receiving oxygen and essential nutrients, causing brain cells to begin dying within minutes. Stroke is considered one of the leading causes of mortality and longterm disability worldwide. Early and accurate diagnosis is therefore crucial for improving patient survival rates and minimizing permanent neurological damage.

In clinical practice, medical imaging plays a fundamental role in stroke diagnosis. Among the available imaging modalities, CT scans are widely used as the first diagnostic tool in emergency situations because they are fast, widely available, and capable of identifying brain abnormalities such as hemorrhage, infarction, or tissue damage. Despite these advantages, the manual interpretation of CT brain scans remains a challenging and time-consuming process. Radiologists must carefully analyze subtle variations in tissue density, grayscale intensity, and structural patterns within the brain. Early-stage stroke symptoms may appear as extremely small or faint abnormalities, making them difficult to detect even for experienced specialists.

Recent advancements in Artificial Intelligence (AI) and Deep Learning (DL) have significantly transformed the field of medical image analysis. In particular, CNNs have demonstrated remarkable success in automatically learning meaningful features from complex image data. CNN-based systems can analyze pixel-level information and progressively extract hierarchical representations that help identify

disease patterns in medical images. Among various CNN architectures, Residual Networks (ResNet) have proven highly effective for image classification tasks.

The ResNet-50 model introduces residual learning through skip connections that allow gradients to flow efficiently through deep neural networks. This architecture overcomes the vanishing gradient problem and enables the network to learn complex hierarchical patterns from images. Due to its powerful feature extraction capability, ResNet-50 has been widely adopted in medical imaging applications, including tumor detection, organ segmentation, and disease diagnosis.

However, while deep architectures like ResNet-50 are capable of learning high-level semantic features, they may sometimes lose very fine spatial details due to repeated downsampling operations and large receptive fields. In medical images such as CT scans, important indicators of stroke may appear as subtle texture variations or small intensity differences that are difficult to capture using deep networks alone.

To address this challenge, lightweight CNN architectures can be incorporated to capture finegrained spatial information. SqueezeNet is a compact convolutional neural network that utilizes Fire modules, consisting of squeeze and expand layers, to efficiently learn detailed spatial features while maintaining a small number of parameters. SqueezeNet is particularly effective at capturing local textures, subtle intensity variations, and small structural patterns that may complement the deep hierarchical features extracted by larger networks.

In the proposed system, ResNet-50 is used as the primary backbone network responsible for deep feature extraction from CT brain images. The extracted feature maps are then passed through a SqueezeNet-inspired Fire module that acts as a feature refinement block. This module applies squeeze and expand convolution operations to refine the deep features obtained from ResNet-50 and capture additional local spatial patterns. The refined feature maps are then processed by global pooling and classification layers to produce the final stroke prediction.

1.2 Problem Statement

Stroke diagnosis is an extremely time-sensitive process in clinical practice. Delays in detecting stroke conditions can lead to severe neurological damage, permanent disability, or even death. Although CT scans are widely used for stroke diagnosis, manual interpretation of these images requires significant expertise and can be time-consuming. Radiologists must analyze subtle visual patterns such as minor changes in tissue density, small lesions, and irregular brain structures, which may not always be clearly visible.

Traditional computer-aided diagnostic systems based on classical machine learning techniques rely heavily on manually designed feature extraction methods. These handcrafted features often fail to capture the complex and heterogeneous patterns present in CT brain images, resulting in limited accuracy and poor generalization.

Deep learning models have improved automated image analysis by learning features directly from data. However, relying solely on a single deep architecture may still present limitations. Deep networks like ResNet-50 focus primarily on learning high-level semantic patterns but may overlook fine-grained spatial details due to repeated downsampling and large receptive fields. Conversely, lightweight CNN architectures can capture local textures and subtle intensity variations but may lack the depth required to understand complex structural relationships within medical images.

Therefore, there is a need for an intelligent system that can combine the strengths of deep hierarchical feature extraction and fine-grained spatial analysis. Such a system should be capable of identifying both large-scale structural abnormalities and subtle texture variations within CT scans in order to achieve more accurate stroke detection. This project addresses the problem by developing an automated brain stroke detection system that uses ResNet-50 as the primary deep learning backbone while incorporating a SqueezeNet-inspired Fire module as a feature refinement block. By refining the deep features extracted from the backbone network, the system aims to improve classification accuracy and provide reliable diagnostic support.

1.3 Objectives

The primary objective of this project is to develop an intelligent and automated deep learning system capable of detecting brain stroke from CT scan images with high accuracy and reliability. The system aims to assist healthcare professionals by providing rapid diagnostic support through the use of advanced convolutional neural network architectures. The objectives of the proposed system are described as follows.

1.3.1 Development of an Automated Stroke Detection System

The first objective is to design and implement an automated system capable of detecting brain stroke from CT scan images using deep learning techniques. The system should reduce dependency on manual interpretation and provide quick diagnostic assistance to medical professionals.

1.3.2 Utilization of ResNet-50 as the Primary Backbone Network

Another important objective is to utilize the ResNet-50 architecture as the primary convolutional neural network for extracting deep hierarchical features from CT brain images. The residual learning mechanism of ResNet-50 enables the network to capture complex structural patterns in medical images while maintaining stable training performance.

1.3.3 Integration of SqueezeNet for Feature Refinement

The system aims to incorporate a SqueezeNet-inspired Fire module as a feature refinement mechanism to capture fine-grained spatial details from CT images. This module helps detect subtle textures, small lesions, and local intensity variations that complement the deep semantic features extracted by ResNet-50.

1.3.4 Sequential Feature Refinement Architecture

Another objective is to design a sequential architecture in which CT brain images are first processed by the ResNet-50 backbone for deep feature extraction. The resulting feature maps are then refined using a SqueezeNet-inspired Fire module that enhances spatial detail representation.

1.3.5 Feature Refinement for Enhanced Representation

The Fire module processes the extracted feature maps using squeeze and expand convolution layers. This refinement process allows the model to capture additional spatial information and produce a richer feature representation for stroke classification.

1.3.6 Improvement of Classification Accuracy

The system aims to improve the overall stroke detection accuracy by leveraging complementary feature extraction strategies. By refining deep hierarchical features with additional spatial feature learning, the model can better identify subtle abnormalities present in CT scans.

1.3.7 Evaluation of Model Performance

Another objective is to evaluate the performance of the proposed model using standard evaluation metrics such as accuracy, precision, recall, and F1-score. These metrics help assess the reliability and effectiveness of the stroke detection system.

1.3.8 Development of a Web-Based Diagnostic Platform

The final objective is to integrate the trained deep learning model into a web-based application that enables clinicians to upload CT scan images and obtain diagnostic predictions quickly. This platform ensures practical usability of the system in real clinical environments.

CHAPTER 2

SYSTEM ANALYSIS

2.1 Literature Survey

The literature review conducted during Stage-I examined several research studies related to automated brain stroke detection using machine learning (ML) and DL techniques. The primary objective of this review was to understand how medical imaging data, particularly CT brain scans, can be analyzed using AI-based models to identify stroke conditions effectively. Various studies highlighted the importance of CNN architectures in extracting meaningful features from complex medical images and demonstrated that DL models outperform traditional ML approaches in medical diagnosis tasks [1], [2].

The literature review analyzed different categories of approaches, including traditional ML techniques, CNN-based models, and hybrid deep learning architectures. These studies collectively showed significant advancements in automated stroke detection systems while also identifying certain limitations. These limitations provided motivation for improving the system architecture in the proposed work.

2.1.1 Traditional Machine Learning Approaches

Early research in automated stroke detection primarily relied on ML algorithms combined with handcrafted feature extraction techniques. In these systems, CT or MRI brain images were first processed using image processing methods to extract features representing texture, intensity distribution, and structural patterns. Common feature extraction techniques included Gray Level Co-occurrence Matrix (GLCM), Local Binary Patterns (LBP), and Histogram of Oriented Gradients (HOG). These handcrafted features were then classified using ML algorithms such as Support Vector Machine (SVM), K-Nearest Neighbors (KNN), Decision Tree, Random Forest, and Naïve Bayes classifiers [5].

Although these approaches showed moderate success in identifying stroke patterns, they suffered from several limitations. The performance of these models was highly dependent on the quality of manually engineered features. Designing such features

required significant domain expertise and extensive preprocessing. Furthermore, handcrafted features often failed to capture complex spatial relationships present in medical images. Studies reported that ML-based stroke detection systems achieved relatively lower accuracy when applied to large and diverse CT datasets [5]. These limitations encouraged researchers to explore DL-based methods capable of automatically learning relevant features directly from medical images.

2.1.2 Deep Learning Based Stroke Detection

Recent research has shown that DL models, particularly CNN architectures, have become highly effective for medical image analysis. Unlike traditional ML methods, CNN models automatically learn hierarchical feature representations from images through multiple convolutional layers. Several researchers have applied transfer learning using pre-trained CNN models to stroke detection tasks. Models such as ResNet have been widely used due to their strong feature extraction capabilities [3], [6].

Among these architectures, ResNet has received considerable attention for medical imaging applications. The introduction of skip connections in ResNet enables the training of very deep neural networks by allowing gradients to flow efficiently through multiple layers. This mechanism helps overcome the vanishing gradient problem and improves model stability during training [6].

For example, Hou et al. (2021) demonstrated the effectiveness of deep learning models in detecting early stroke regions in CT brain scans. Their research showed that deep CNN models could identify subtle stroke patterns more accurately than traditional approaches [1].

Similarly, Tasci et al. (2023) applied deep learning techniques using CT imaging for automated ischemic stroke detection and reported high classification accuracy, confirming the effectiveness of CNN-based approaches [2].

These findings indicate that deep CNN models provide strong performance for stroke detection tasks. However, despite their effectiveness, deep CNN architectures may sometimes overlook fine spatial details due to repeated downsampling operations. This

limitation suggests that incorporating additional mechanisms to capture fine-grained features could further improve model performance.

2.1.3 Hybrid Deep Learning Models

To address the limitations of single-architecture models, recent studies have explored hybrid DL architectures that combine multiple neural network components for improved feature representation. In many hybrid architectures, different CNN models process the same input image and extract complementary features which are later combined using fusion techniques. These approaches have shown promising improvements in classification accuracy.

For instance, Kulathilake et al. (2025) proposed a hybrid deep learning model for brain stroke detection using medical imaging. Their results showed that integrating multiple feature extraction methods improved model accuracy and reduced overfitting [4].

Similarly, Gheibi et al. (2023) developed a residual CNN-based framework for medical image classification that demonstrated improved performance by combining deep feature extraction with enhanced learning strategies [3].

These studies suggest that hybrid architectures allow different models or modules to specialize in extracting different types of information. Deep CNN models typically capture high-level semantic features and global structural patterns, while lightweight modules can capture fine spatial textures and local intensity variations more effectively. By combining deep feature extraction with additional feature refinement mechanisms, hybrid models can achieve more robust performance in medical image classification tasks.

Table 2.1 : Literature Survey

Ref	Authors & Year	Title / Source	Model / Methodology	Dataset	Key Findings / Results
[1]	X. Hou et al., 2021	Deep learning based early detection of stroke using CT brain images (IEEE Access)	Deep CNN model	CT brain images	Demonstrated early stroke detection with high accuracy; deep learning outperformed traditional methods

[2]	B. Tasci et al., 2023	Automated detection of ischemic stroke using deep learning (BSPC)	CNN + Transfer Learning	CT imaging dataset	Achieved high classification accuracy; effective for ischemic stroke detection
[3]	M. Gheibi et al., 2023	Deep residual CNN for medical image classification (CMPB)	Residual CNN (ResNet-based)	Medical imaging dataset	Improved classification performance using deep residual learning
[4]	S. Kulathilake et al., 2025	Hybrid deep learning models for stroke detection (IJMI)	Hybrid CNN architecture	Medical image dataset	Combined models improved feature extraction and reduced overfitting
[5]	S. Raj et al., 2025	AI-based systems for stroke diagnosis (JHE)	Traditional ML + AI	CT brain scans	Highlighted limitations of ML and importance of automated AI systems
[6]	K. He et al., 2016	Deep Residual Learning (CVPR)	ResNet architecture	ImageNet dataset	Introduced residual learning to train deep networks effectively
[7]	F. Iandola et al., 2016	SqueezeNet (arXiv)	Lightweight CNN (Fire modules)	Image dataset	Achieved high accuracy with fewer parameters and reduced complexity
[8]	F. Chollet, 2015	Keras Deep Learning Library	Framework	—	Simplified implementation of deep learning models
[9]	M. Abadi et al., 2016	TensorFlow Framework	Framework	—	Enabled scalable deep learning model training
[10]	Kaggle Dataset	Brain Data Organised Dataset	Dataset	CT brain images	Provided structured dataset for model training and evaluation

2.2 Dataset Description

The dataset used in this project consists of brain CT scan images that are used to detect the presence of stroke using deep learning techniques. These images represent different conditions of the brain, including normal cases and stroke-affected regions. The dataset plays a crucial role in training and evaluating the proposed deep learning model for accurate stroke detection. Each image in the dataset represents a CT scan of a patient's brain. These images contain important visual patterns such as intensity variations, abnormal tissue regions, and structural differences, which are analyzed by the deep learning model to identify stroke conditions. The dataset enables the model to learn complex features directly from images without the need for manual feature extraction.

2.2.1 Dataset Source

The dataset used in this project was obtained from publicly available medical imaging repositories and Kaggle, which provide datasets for research and experimentation in medical image analysis. The dataset, commonly referred to as the Brain CT Stroke Dataset, contains CT scan images collected from medical imaging studies. These images are labeled to indicate whether a stroke is present or not, allowing the model to perform supervised learning and accurately classify brain conditions

2.2.2 Number of Samples

The dataset consists of multiple CT scan images categorized into two primary classes, namely normal brain images and stroke-affected brain images. The dataset is structured to maintain balance between the two classes to ensure unbiased model training. Each image serves as an input sample for the deep learning model, enabling it to learn distinguishing features between stroke and non-stroke cases effectively.

2.2.3 Features

Unlike tabular datasets, the features in this project are extracted directly from CT scan images using a hybrid deep learning model. The ResNet50 component extracts high-level hierarchical features such as structural patterns and abnormalities in brain regions. These extracted features are then passed to the SqueezeNet component, which

refines the feature maps by capturing fine spatial details and reducing redundancy. This sequential feature extraction process allows the model to learn both global and local patterns present in CT images, thereby improving the accuracy of stroke detection.

2.2.4 Data Format

The dataset is stored in image formats such as JPG or PNG, where each file represents a CT scan of the brain. During processing, these images are converted into numerical arrays so that they can be used as input for the deep learning model. Before training, the images undergo preprocessing steps such as resizing, normalization, and conversion into tensors. These steps ensure that all images are consistent in size and format, making them suitable for efficient model training.

2.2.5 Data Preprocessing

Data preprocessing is an essential step in preparing the CT images for training the model. The preprocessing process involves resizing the images to a fixed dimension, normalizing pixel intensity values, and converting images into numerical arrays. These operations help maintain uniformity in the dataset and enhance the learning capability of the model by reducing variations caused by different image sizes and intensity scales.

2.2.6 Data Cleaning

Data cleaning is performed to ensure the quality and reliability of the dataset. During this process, invalid or corrupted images are removed, duplicate images are eliminated, and all samples are verified for correct labeling. This ensures that the model is trained using only high-quality data, which improves prediction accuracy and reduces the chances of errors during classification.

2.2.7 Train–Test Split

To evaluate the performance of the model, the dataset is divided into training and testing subsets. The training dataset is used to train the model, while the testing dataset is used to evaluate its performance on unseen data. The dataset is divided using an 80:20 ratio, where 80 percent of the data is used for training and 20 percent is used for testing. This split ensures that the model generalizes well and performs accurately on new CT images.

2.3 System Specifications

2.3.1 Hardware

Table 2.2 : Hardware Requirements

Category	**Requirement**
Processor (CPU)	Intel i5 / Ryzen 5 or higher
RAM	8–16 GB
Storage	128 GB or higher
GPU (Optional)	NVIDIA GPU (for faster training)

2.3.2 Software

Table 2.3 : Software Requirements

Category	**Requirement**
Operating System	Windows / Linux
Programming Language	Python 3.x
Backend Framework	Django
Frontend	React, HTML, CSS, JavaScript
Deep Learning	TensorFlow / Keras
Libraries	NumPy, OpenCV, Pandas
Database	SQLite / MySQL

2.4 Technical Feasibility

2.4.1 Dataset Feasibility

The dataset used in this project consists of CT scan images, which are widely used for detecting brain abnormalities. It is suitable for deep learning-based classification as it provides labeled image data for supervised learning. Unlike tabular data, it enables the model to learn spatial patterns directly from images. The dataset is divided into two classes for binary classification of stroke and non-stroke cases.

2.4.2 Hardware Adequacy

The model training can be performed using standard computing resources. While CPU-based systems can handle moderate datasets, the use of GPU (such as Kaggle cloud GPU) significantly accelerates training and reduces computation time.

2.4.3 Model Selection Justification

The model used in this project is a hybrid deep learning architecture that combines ResNet50 and SqueezeNet for efficient and accurate brain stroke detection from CT scan images. This hybrid approach is designed to leverage the strengths of both architectures in a sequential manner. ResNet50 serves as the primary feature extraction backbone, enabling the model to learn complex hierarchical features while overcoming the vanishing gradient problem through residual connections. Following this, SqueezeNet is used to refine and compress the extracted features by capturing fine spatial details and reducing redundancy. This combination enhances feature representation and improves classification accuracy. The hybrid architecture provides a balance between performance and efficiency, making it highly suitable for medical image analysis.

2.4.4 Training Feasibility

The model is trained successfully using appropriate hyperparameters such as batch size, epochs, optimizer, and loss function. The training process shows stable convergence with increasing accuracy and reliable prediction performance for stroke detection.

CHAPTER 3

SYSTEM DESIGN

3.1 Architecture

The architecture of the proposed brain stroke detection system is designed to automatically analyze CT brain images and classify them into stroke or normal categories. The system uses a deep learning approach in which CT images are processed through convolutional neural network architectures to extract meaningful features and perform classification. The architecture integrates multiple components including image input, preprocessing, deep feature extraction, feature refinement, and final prediction. The system follows a structured workflow beginning with image acquisition and ending with classification output. Each stage in the architecture contributes to improving the accuracy and reliability of the stroke detection model.

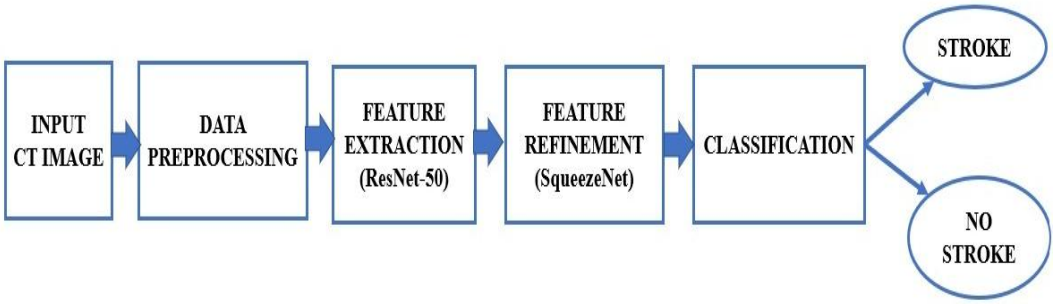

Figure 3.1 : System Architecture

3.1.1 Input Layer

The first stage of the system architecture is the input layer, where CT brain images are provided to the system. These images represent brain scan slices obtained from CT imaging procedures. The images serve as the raw input for the deep learning model. Before being processed by the neural network, the images are loaded into the system and converted into numerical arrays that can be interpreted by the model.

3.1.2 Image Preprocessing

Image preprocessing is performed to standardize the input data and ensure compatibility with the neural network architecture. All images are resized to a

uniform resolution of 224 × 224 pixels so that they match the input requirements of the CNN model. Pixel values are normalized to maintain consistent numerical ranges across the dataset. This helps stabilize the training process and improves the learning efficiency of the neural network. Additional preprocessing operations such as augmentation may be applied to improve dataset diversity and reduce overfitting.

3.1.3 Feature Extraction Using ResNet

After preprocessing, the CT images are passed through the primary CNN backbone used for feature extraction. The architecture utilizes ResNet as the main deep learning model responsible for extracting hierarchical image features. ResNet uses resdual connections that allow gradients to flow through deep layers during training. This enables the network to learn complex patterns present in CT images such as structural abnormalities, lesion shapes, and spatial patterns associated with stroke. The output of this stage is a deep feature representation that captures high level information about the input image. These feature maps represent important visual patterns extracted from the CT scan and serve as the input for the next stage of the architecture.

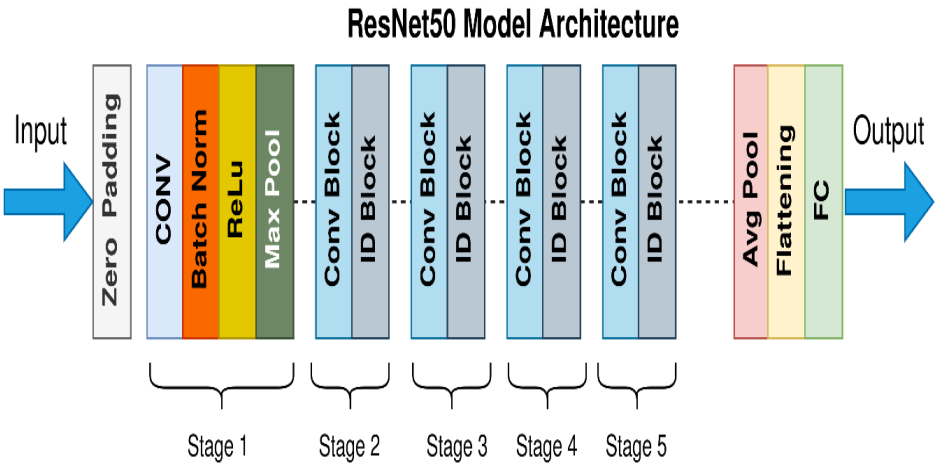

Figure 3.2 : ResNet Architecture

3.1.4 Feature Refinement Using SqueezeNet Fire Module

After the deep features are extracted by ResNet, they are passed to a SqueezeNet-inspired Fire module that acts as a feature refinement block. This module processes the feature maps generated by ResNet rather than processing the original CT image.

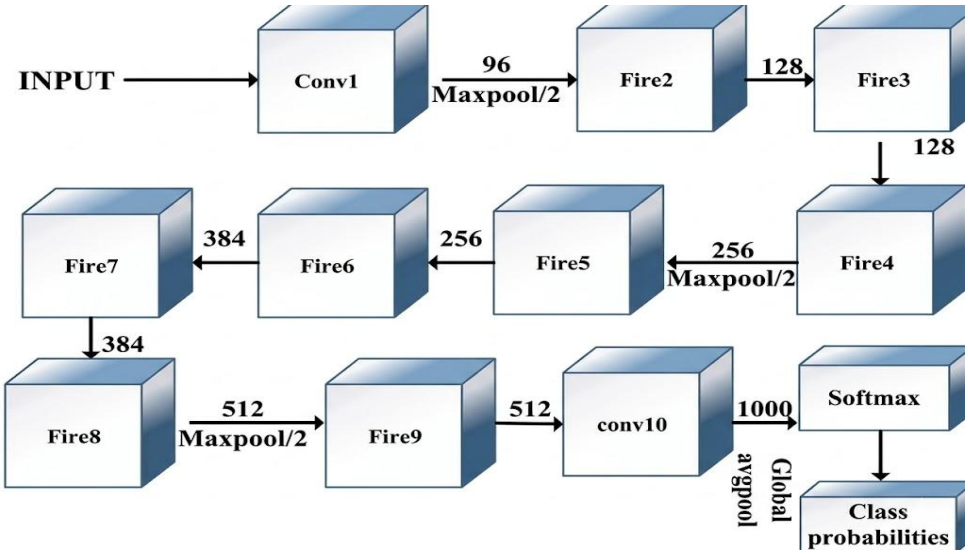

Figure 3.3 : SqueezeNet Architecture

The Fire module consists of a squeeze layer followed by expand layers. The squeeze layer applies 1 × 1 convolution filters to reduce the number of channels in the feature maps. This operation helps compress the feature representation and reduce computational complexity. The compressed features are then passed to the expand layer, which applies parallel 1 × 1 and 3 × 3 convolution filters. These convolution operations refine the extracted features by capturing both channel-wise relationships and local spatial patterns present in the feature maps. Through this refinement process, the Fire module enhances subtle spatial details such as texture variations and intensity differences that may correspond to stroke affected regions in CT images.

3.1.5 Feature Transformation and Pooling

After feature refinement, the resulting feature maps are passed through a global average pooling layer. This operation reduces the spatial dimensions of the feature maps while preserving the most important information learned by the

network. Global average pooling converts the multi-dimensional feature maps into a compact feature vector that summarizes the important patterns detected in the CT image. This transformation reduces the number of parameters and helps prevent overfitting.

3.1.6 Classification Layer

The transformed feature representation is passed to fully connected layers that perform the final classification task. These layers interpret the extracted features and produce a prediction indicating whether the input CT image represents a stroke condition or a normal brain scan. Dense layers are used to learn the relationship between the extracted features and the target classes. A sigmoid activation function is used in the final layer to generate a binary classification output.

3.1.7 Prediction Output

The final stage of the architecture generates the prediction result. The system outputs one of two possible labels:

- Stroke detected
- No stroke detected

The prediction is then displayed through the user interface, allowing medical professionals to interpret the result quickly.

3.2 Module Design

The proposed system is divided into several functional modules, each responsible for a specific task within the stroke detection workflow. The modular design improves system organization and allows each component to be developed and maintained independently.

3.2.1 User Interface Module

The user interface module provides the front-end environment where users interact with the system. Through this interface, users can upload CT scan images and view the prediction results generated by the model. The interface is designed to be simple and user friendly so that medical staff can operate the system without technical complexity.

3.2.2 Image Upload Module

The image upload module allows users to submit CT brain images to the system. The uploaded images are validated to ensure that they are in the correct format before being processed by the deep learning model. Once validated, the images are forwarded to the preprocessing module.

3.2.3 Preprocessing Module

The preprocessing module prepares the uploaded images for analysis by the neural network. This module performs operations such as resizing, normalization, and image formatting to ensure that the data matches the input requirements of the model. Proper preprocessing improves the reliability and consistency of the prediction process.

3.2.4 Feature Extraction Module

The feature extraction module uses the deep learning architecture to analyze CT images and extract meaningful patterns. In this system, the CT image is first processed by the ResNet backbone network, which extracts deep hierarchical features representing structural abnormalities and spatial relationships in the brain. The feature maps generated by ResNet are then passed through a SqueezeNet-inspired Fire module that refines these features by capturing additional spatial patterns, texture variations, and intensity differences. This sequential feature extraction and refinement process enables the model to obtain a richer representation of the CT image before classification.

3.2.5 Classification Module

The classification module interprets the extracted features and generates the final prediction. The refined feature representation is passed through dense layers that classify the image as either stroke affected or normal.

3.2.6 Result Display Module

The result display module presents the prediction output to the user. The system displays the classification result along with the uploaded CT image so that users can easily interpret the outcome.

3.3 UML Diagrams

UML diagrams are used to represent the structural and behavioral design of the proposed system. These diagrams provide a visual representation of how different components of the system interact with each other. They help in understanding the overall architecture and workflow of the system in a clear and organized manner. UML diagrams are broadly categorized into structural diagrams and behavioral diagrams, each serving a specific purpose. Structural diagrams illustrate the static aspects of the system, such as classes, objects, and their relationships. Behavioral diagrams, on the other hand, depict the dynamic behavior of the system, including interactions and processes over time.

3.3.1 Use Case Diagram

The use case diagram illustrates the interaction between the user and the system functionalities. It shows that the user can register, log in, upload CT images, preprocess the images, and obtain stroke prediction results. The diagram highlights that all system functionalities are user-driven and emphasizes the main goal of the system, which is to assist users in detecting brain stroke conditions efficiently. It provides a high-level overview of how the system behaves from the user's perspective.

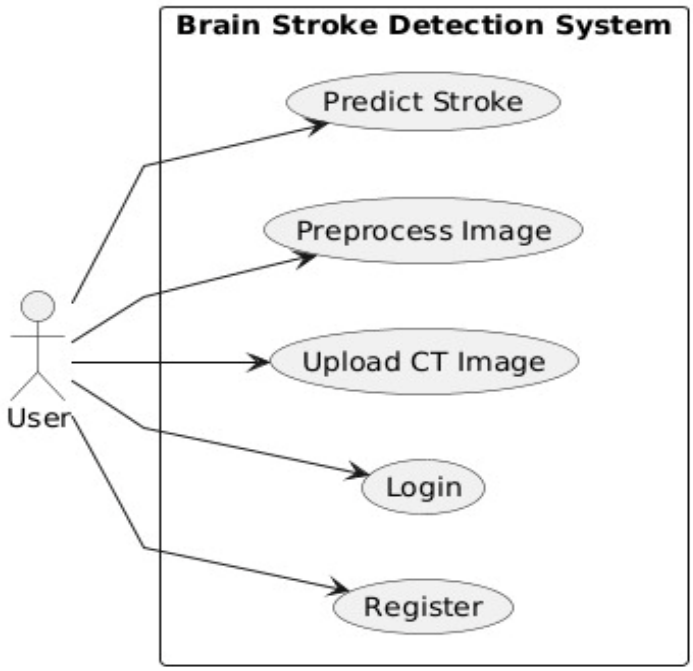

Figure 3.4 : Usecase Diagram

3.3.2 Class Diagram

The class diagram represents the structural design of the Brain Stroke Detection System by defining the main classes and their relationships. The User class handles authentication operations such as login and registration. The Image Uploader class is responsible for receiving CT scan images from the user and forwarding them to the processing pipeline. The Preprocessor class performs essential preprocessing tasks to prepare the image for analysis. The Model class represents the deep learning model, specifically ResNet50 in your project, which extracts meaningful features from the CT images. These extracted features are passed to the Predictor class, which applies a threshold-based decision mechanism to determine whether a stroke is present. Finally, the Result class stores and displays the prediction outcome to the user.

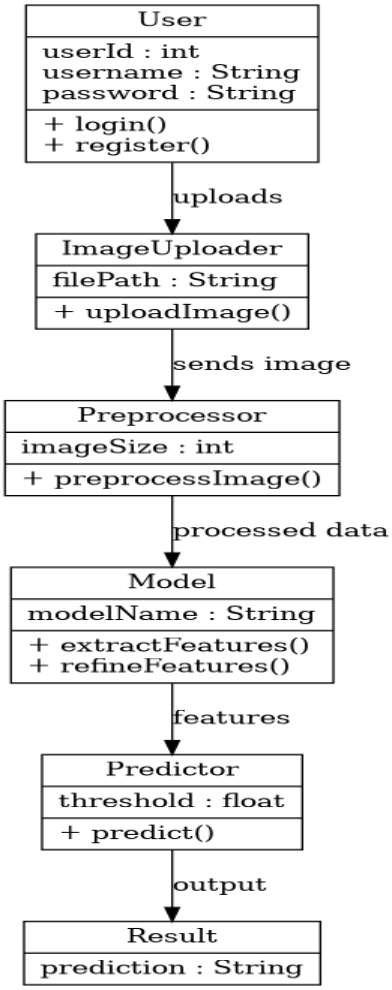

Figure 3.5 : Class Diagram

3.3.3 Sequence Diagram

The sequence diagram illustrates the interaction between different system components over time. It shows how the user uploads an image through the UI, which is then sent to the backend. The backend forwards the image to the preprocessor, which processes the image and sends it to the model for feature extraction. The model refines the features and passes them to the predictor, which generates the result. The result is then sent back through the backend to the UI and displayed to the user. This diagram clearly represents the dynamic behavior and message flow within the system.

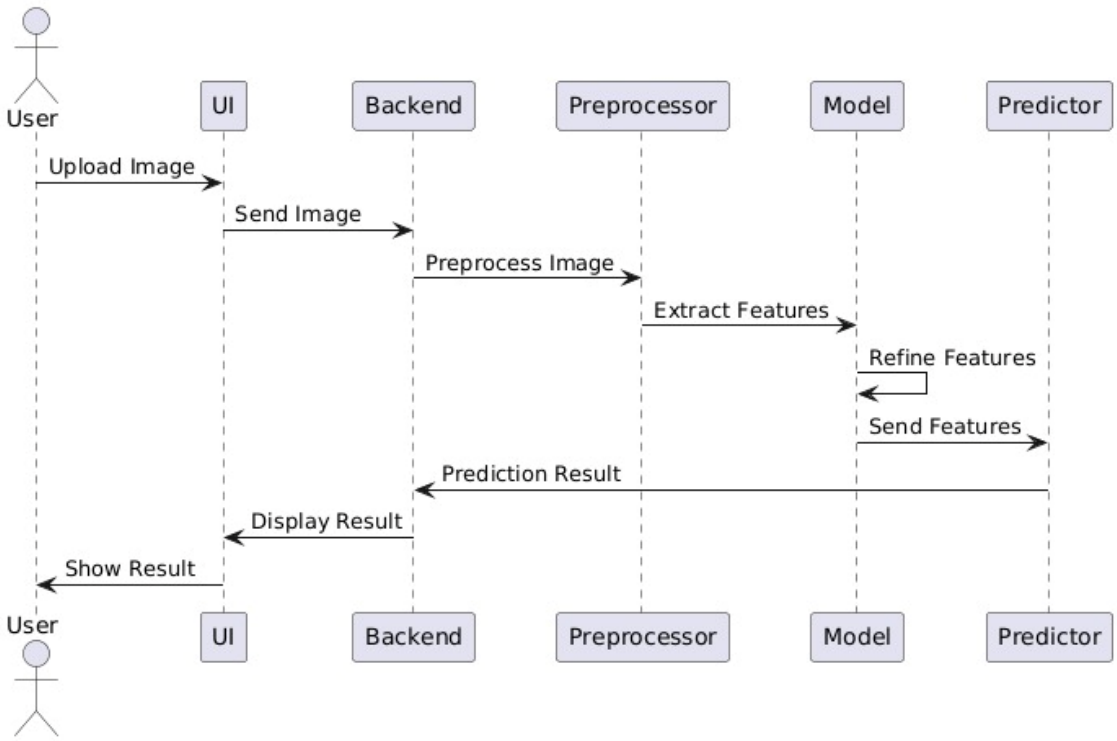

Figure 3.6 : Sequence Diagram

3.3.4 Activity Diagram

The Activity Diagram illustrates the step-by-step workflow of the stroke detection system from user interaction to result generation. It begins with the user opening the application, followed by a decision point to either register as a new user or log in. After authentication, the user uploads a CT image, which is then sent to the backend for processing. The system performs preprocessing, feature extraction, and refinement before predicting the presence of a stroke. Finally, the generated result is displayed to the user, completing the process flow.

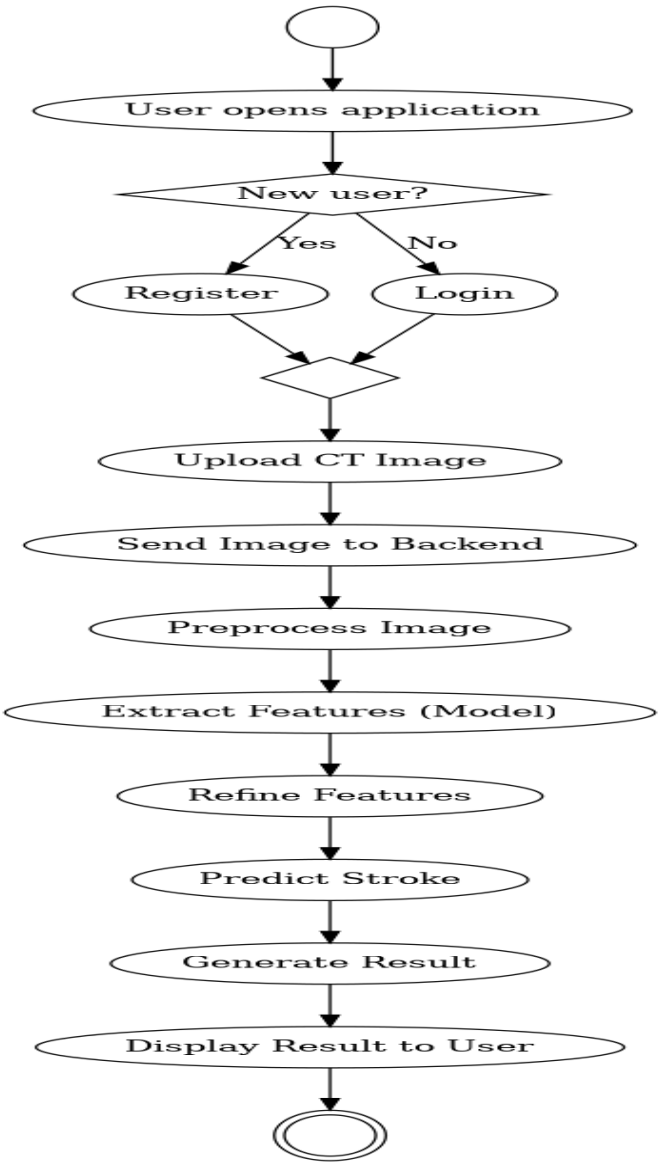

Figure 3.7 : Activity Diagram

CHAPTER 4
IMPLEMENTATION

4.1 Technologies Used

The implementation of the proposed brain stroke detection system involves the use of several technologies that support deep learning model development, data processing, and system deployment. These technologies provide the tools required to train the neural network model, process CT scan images, and build a web-based platform through which users can interact with the system. The major technologies used in the implementation are described below.

4.1.1 Programming Language

The primary programming language used for developing the system is Python. Python is widely used in machine learning and deep learning applications because of its simplicity, readability, and extensive ecosystem of scientific computing libraries. It provides efficient support for image processing, numerical computation, and neural network implementation. Python allows seamless integration of deep learning frameworks, data processing libraries, and web development tools. This makes it suitable for building end-to-end machine learning systems that involve data preprocessing, model training, and application deployment.

4.1.2 Deep Learning Frameworks

The deep learning model used in this project is implemented using TensorFlow and Keras. These frameworks provide powerful tools for designing, training, and evaluating neural network architectures.

TensorFlow is a widely used open-source deep learning framework that supports efficient computation of large-scale machine learning models. It provides optimized operations for training neural networks and supports GPU acceleration when available.

Keras is a high-level neural network API built on top of TensorFlow. It simplifies the process of building deep learning models by providing easy-to-use functions for defining layers, compiling models, and managing training processes. Using Keras, complex architectures such as residual networks can be implemented with relatively simple code.

In addition to these frameworks, Scikit-learn is used for model evaluation and data analysis tasks. It provides useful utilities for generating performance metrics such as accuracy, precision, recall, and confusion matrices.

4.1.3 Database

The proposed system does not require a complex database management system because the primary data used in the project consists of CT scan image files stored in the dataset directory. These image files are directly accessed by the model during training and prediction. However, for the web application component, a lightweight database such as SQLite may be used to store user information, uploaded images, or prediction history. SQLite provides a simple and efficient database solution that integrates easily with backend frameworks.

4.1.4 Frontend Framework

The user interface of the system is implemented using React. React is a widely used frontend library for building interactive and responsive web interfaces. It allows developers to create reusable user interface components and manage application state efficiently. Using React, a simple interface is created that allows users to upload CT scan images and view prediction results generated by the deep learning model. The frontend communicates with the backend server through API requests, enabling the system to process uploaded images and return classification results.React improves the usability of the system by providing a clean and responsive interface that allows users to interact with the application easily.

4.2 Algorithm

The proposed brain stroke detection system is implemented using a deep learning–based image classification model. The model analyzes CT brain scan images and classifies them into two categories: stroke and normal. The system uses

a convolutional neural network architecture in which image features are automatically learned from the dataset. The architecture integrates two neural network models that operate together to improve feature extraction and classification performance.

4.2.1 Algorithm Used

The primary algorithm used in this system is a convolutional neural network. Convolutional neural networks are widely used for image classification tasks because they are capable of learning spatial features directly from image data. In this project, the model architecture is based on transfer learning using a deep residual network as the main feature extraction backbone. The residual network is responsible for learning hierarchical features from CT images. In addition, a lightweight convolutional network is used in parallel to refine feature representations and capture fine spatial patterns. The outputs from both networks are combined before performing the final classification. This hybrid deep learning approach enables the system to analyze both global structural patterns and fine spatial variations present in CT brain images.

4.2.2 Justification for Model Selection

The convolutional neural network architecture was selected because it is highly effective for image analysis tasks. Unlike traditional machine learning algorithms, convolutional neural networks automatically learn feature representations directly from image data, eliminating the need for manual feature extraction.

The residual network architecture was chosen as the main backbone because it allows very deep networks to be trained efficiently using residual connections. These connections help prevent gradient degradation during training and allow the network to learn complex hierarchical patterns present in CT brain images.

The additional lightweight network is used to capture fine spatial details such as subtle texture variations and small intensity differences that may not be fully captured by deeper networks alone. Combining these two networks improves the richness of the extracted features and enhances classification performance. This model architecture is also suitable for the dataset used in the project because transfer

learning allows pre-trained networks to adapt effectively to smaller medical image datasets.

4.2.3 Working Principle

The working principle of the proposed model involves several stages that transform the input CT image into a classification result. First, the CT brain image is provided as input to the system. The image is preprocessed by resizing it to a fixed resolution and normalizing the pixel values. This ensures that all images are in a consistent format suitable for neural network processing.

After preprocessing, the image is passed through the convolutional neural network architecture. Convolutional layers apply filters to the image in order to extract important visual features. These filters detect patterns such as edges, textures, shapes, and structural variations in the brain image. The deep residual network processes the image to learn high-level structural patterns and spatial relationships within the CT scan. At the same time, the lightweight convolutional network processes the same image to capture fine spatial details such as subtle texture variations and small abnormalities. The features extracted by both networks are combined into a single feature representation. This fused representation contains both global structural information and refined local details.

Finally, the combined feature vector is passed through fully connected layers that perform the classification task. The output layer produces a prediction indicating whether the input image represents a stroke condition or a normal brain scan.

4.2.4 Mathematical Framework

The convolutional neural network model operates through a sequence of mathematical operations that transform the input image into feature representations and classification outputs. The main operation in convolutional neural networks is the convolution operation. In convolution, a small filter matrix is applied across the input image to extract spatial features.

The convolution operation can be represented as Feature Map = Input Image $*$ Filter + Bias x After convolution, a non-linear activation function is applied to introduce nonlinearityinto the model. One commonly used activation function is the rectified

linear unit, the model. One commonly used activation function is the rectified linear unit, defined as $ReLU(x) = \max(0, x)$.

This function ensures that negative values are replaced with zero while positive values remain unchanged. Pooling operations are also used to reduce the spatial dimensions of feature maps while preserving important information. This reduces computational complexity and helps the model focus on dominant features. During the final stage, the extracted features are passed through dense layers that compute a weighted combination of the feature values. The final output layer uses a sigmoid activation function to generate a probability value between zero and one. Prediction $= 1 / (1 + e^{(-z)})$ where z represents the weighted sum of the input features.

If the output probability is greater than a predefined threshold, the image is classified as stroke; otherwise, it is classified as normal. Through these mathematical operations, the convolutional neural network learns to identify complex patterns within CT brain images and perform accurate stroke classification.

4.3 Training Process

The training process is a critical stage in developing the deep learning model for brain stroke detection. During training, the model learns to identify patterns from CT brain images and associate those patterns with the correct class labels. The training pipeline involves multiple steps including preparing the dataset, configuring the model architecture, selecting appropriate hyperparameters, executing the training process, monitoring performance metrics, and saving the trained model for later use in prediction.

4.3.1 Data Preparation

The first step in the training process involves preparing the dataset for model training. The CT brain images are organized into labeled directories corresponding to their class categories. Each image is loaded into the system and converted into numerical arrays so that it can be processed by the neural network.

Before feeding the images into the model, preprocessing operations are applied to ensure consistency across the dataset. All images are resized to a fixed resolution of 224 × 224 pixels to match the input requirements of the neural network architecture.

Resizing ensures that every image has the same dimensions, which allows the model to process the dataset efficiently. Pixel values are normalized to maintain a consistent numerical range. Normalization helps stabilize the training process and allows the neural network to converge more effectively during learning.

Data augmentation techniques are also applied to improve dataset diversity and reduce the risk of overfitting. These techniques include image transformations such as rotation, horizontal flipping, zooming, and shifting. Augmentation allows the model to learn robust features by exposing it to different variations of the same image. The dataset is then divided into training and testing subsets. The training set is used to train the model, while the testing set is used to evaluate the model's performance on unseen data.

4.3.2 Model Configuration

After dataset preparation, the system configures a sequential deep learning model for stroke detection. CT scan images are first processed using a ResNet-based backbone to extract high-level features such as structural patterns and abnormalities, with residual connections improving performance. The extracted features are then refined by a secondary convolutional network to capture fine details like textures and edges. Finally, fully connected layers classify the image as stroke or normal, ensuring accurate and stable prediction through progressive feature enhancement.

4.3.3 Hyperparameter Settings

Hyperparameters are configuration parameters that control the training process of the neural network. Selecting appropriate hyperparameters is important for achieving good model performance. The training process uses parameters such as the learning rate, batch size, and number of epochs. The learning rate determines how quickly the model updates its weights during training. A moderate learning rate ensures stable training and prevents the model from diverging.

The batch size determines the number of images processed in each training iteration. Using batches improves computational efficiency and allows the model to update weights gradually. The number of epochs defines how many times the entire training dataset is processed by the model. Increasing the number of epochs allows the model

to learn more complex patterns from the data, although excessive training may lead to overfitting.

4.3.4 Training Execution

Once the model architecture and hyperparameters are defined, the training process is executed. During training, batches of images are fed into the neural network, and the model generates predictions for each image. The predicted outputs are compared with the actual class labels, and a loss value is calculated to measure the difference between predicted and true values. The training algorithm then adjusts the model weights in order to minimize the loss. This process is repeated across multiple epochs until the model gradually learns meaningful features from the dataset. As training progresses, the model improves its ability to correctly classify CT brain images.

4.3.5 Performance Monitoring

During training, the model's performance is continuously monitored using evaluation metrics. Common metrics include accuracy and loss values calculated for both training and validation datasets. Accuracy measures the percentage of images correctly classified by the model, while loss indicates how far the predicted outputs are from the actual labels. Monitoring these metrics helps determine whether the model is learning effectively. Visualization tools are often used to plot training curves that show how accuracy and loss change across training epochs. These graphs help identify potential issues such as overfitting or underfitting.

4.3.6 Model Saving

After completing the training process, the trained model is saved for later use. Saving the model allows it to be loaded and used for prediction without repeating the entire training process. The model file contains the learned weights and architecture parameters that were obtained during training. When a new CT image is uploaded through the system interface, the saved model can be loaded to perform stroke classification. Saving the trained model ensures that the system can efficiently perform predictions and can also be deployed as part of the web-based application for real-time stroke detection.

4.4 User Interface Screens

4.4.1 Authentication Module

The login interface of the Brain Stroke Detection system provides a secure and user-friendly mechanism for accessing the application. The page includes input fields for entering the username and password, allowing registered users to authenticate themselves before using the system. A clean and minimal design is maintained to ensure ease of use, with clearly labeled fields and a login button to initiate the authentication process. The navigation bar at the top displays the application title “Brain Stroke AI” along with options such as Home, About, Contact, Login, and Signup, enabling seamless navigation across different sections. Upon entering valid credentials, the system verifies the user details through the backend authentication module and grants access to the main application features, including image upload, prediction, and dashboard analytics. If the user does not have an account, a signup option is provided to register a new account. This authentication mechanism ensures that only authorized users can access the system, thereby maintaining data security and privacy. The overall design emphasizes simplicity, responsiveness, and secure user interaction.

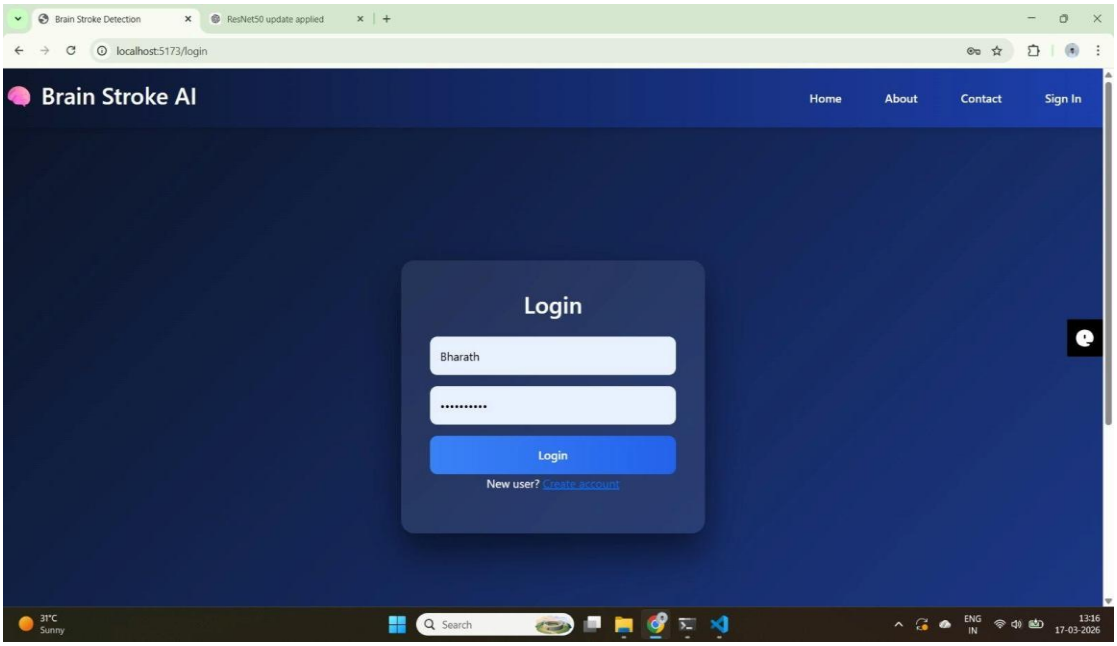

Figure 4.1 : Login Page

4.4.2 User Interface Overview

The home interface of the Brain Stroke Detection system provides a clean and intuitive platform for user interaction. A navigation bar at the top displays the application title “Brain Stroke AI” along with menu options such as Home, About, Contact, Login, and Signup, enabling easy access to different sections. The main section features a visually engaging brain-themed background that aligns with the medical purpose of the application. At the center, a glassmorphism-style panel highlights the core functionality of the system. This panel includes the title “Brain Stroke Detection”, a file upload option to select a CT scan image, and a “Predict Stroke” button to initiate the analysis. The interface is designed to be simple and user-friendly, allowing users to quickly upload images and perform predictions. The modern layout and minimal design enhance usability while effectively integrating the frontend with the underlying deep learning model.

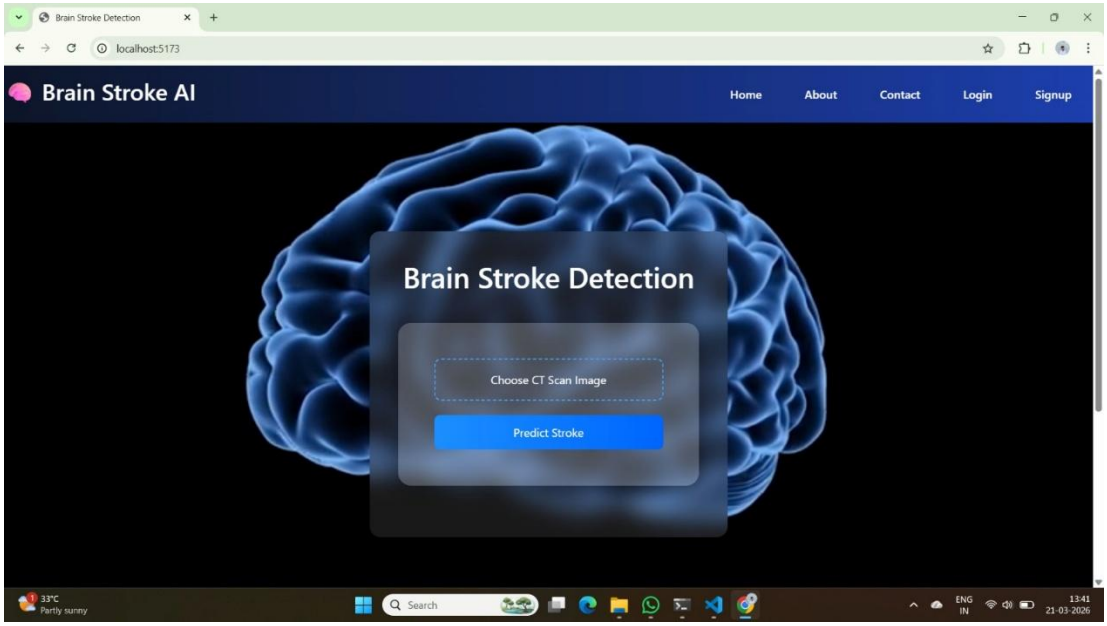

Figure 4.2 : Home Page

4.4.3 Image Upload and Input Handling

This interface illustrates the process of selecting a CT scan image for stroke prediction within the Brain Stroke Detection system. Upon clicking the file upload option, a system dialog window appears, allowing users to browse and choose an image from their local storage. The dialog displays multiple CT scan images categorized into normal and stroke cases, enabling the user to select an appropriate input for analysis. The file selection mechanism ensures flexibility by supporting different image files stored on the system. In the background, the application interface remains visible, showing the upload panel and the "Predict Stroke" button. Once an image is selected, it is uploaded to the system and prepared for processing by the deep learning model. This step represents the initial stage of the prediction pipeline, where user-provided medical images are fed into the system for automated analysis.

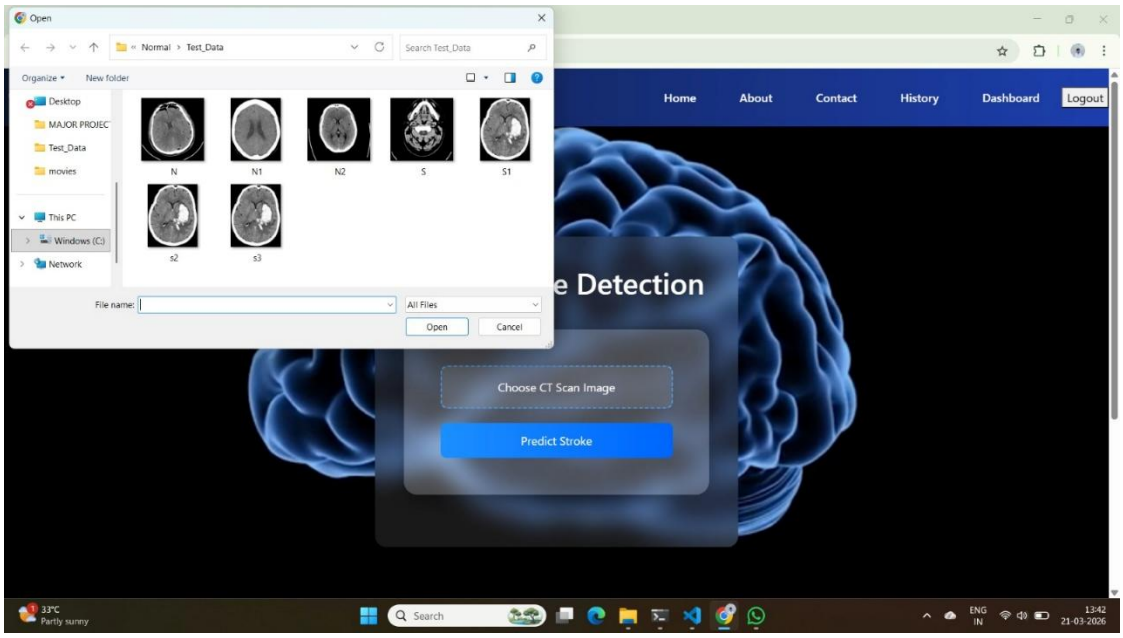

Figure 4.3 : CT Scan Image Selection Interface

4.4.4 Stroke Prediction Result Interface

This interface displays the output of the Brain Stroke Detection system after processing a CT scan image. The navigation bar at the top provides access to different sections such as Home, About, Contact, History, and Dashboard, along with a logout option for authenticated users. The central panel presents the prediction workflow. On the left, the selected CT scan image is shown along with the filename, confirming the input provided by the user. A “Predict Stroke” button is available to initiate the analysis. On the right side, the system displays the prediction result labeled “Stroke”, indicating that the uploaded image has been classified as a stroke case. Alongside this, a Grad-CAM heatmap visualization is presented, highlighting the regions of the image that contributed most to the model’s decision. This interface effectively demonstrates the integration of deep learning predictions with visual explanations, helping users understand not only the result but also the reasoning behind the model’s classification.

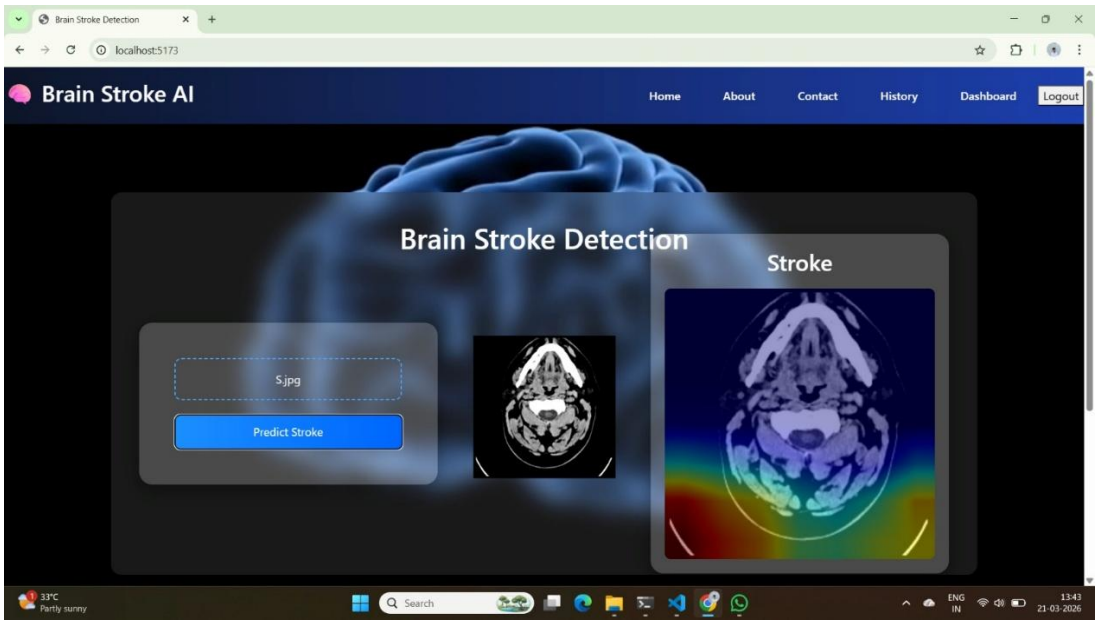

Figure 4.4 : Stroke Prediction Result Interface

4.4.5 About Page Description

The About page provides an overview of the Brain Stroke Detection system and its purpose in assisting medical diagnosis. The interface includes a navigation bar with options such as Home, About, Contact, History, and Dashboard, along with a logout option for authenticated users, ensuring smooth navigation across the application. At the center of the page, the title “Brain Stroke AI” is displayed along with a brief description that explains the system as an AI-powered medical solution designed for rapid and accurate stroke detection from CT scan images using advanced deep learning techniques. It emphasizes the goal of enabling early stroke detection to support healthcare professionals in making timely decisions and improving patient outcomes. It also describes the technologies used, including hybrid deep learning models such as ResNet50 and SqueezeNet, Grad-CAM for interpretability, Django REST API for backend processing, and React for the user interface. Additionally, it highlights key features like real-time prediction, heatmap visualization, prediction history tracking, and secure authentication, providing a comprehensive understanding of the system’s capabilities.

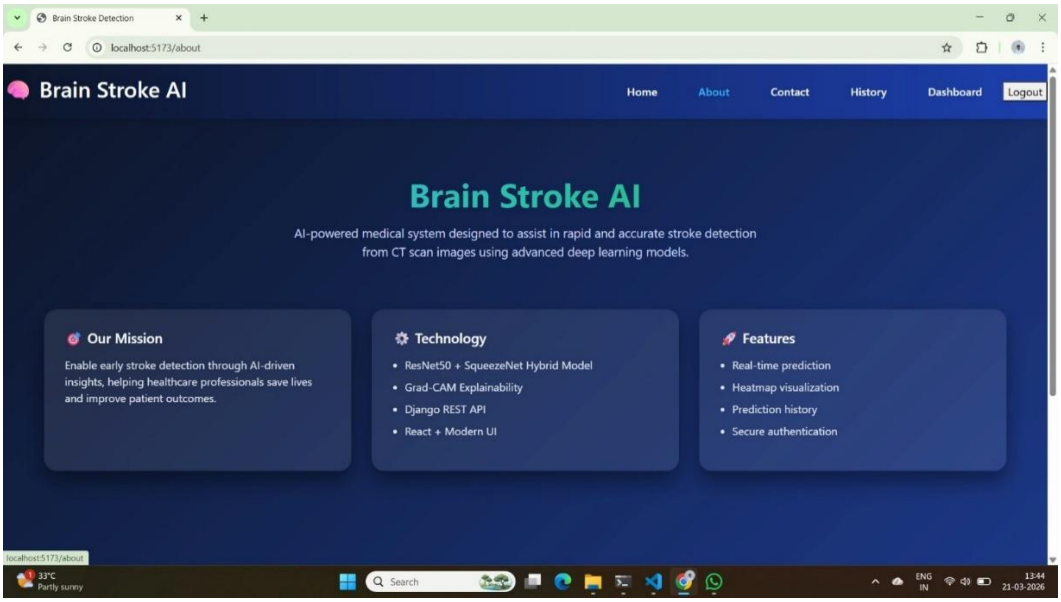

Figure 4.5 : About Page

4.4.6 Contact Module

The Contact page provides a communication interface for users to reach out to the Brain Stroke Detection system team. The navigation bar at the top includes options such as Home, About, Contact, History, and Dashboard, along with a logout option, ensuring easy navigation across the application. At the center, the heading “Contact Us” is displayed along with a short message encouraging users to share their questions or feedback. The page is divided into two main sections, presenting both contact information and a user input form. On the left side, the system displays essential contact details including email, phone number, and location, providing users with direct ways to connect with the team. It also includes a brief statement highlighting the purpose of the system in assisting doctors with faster and more accurate stroke detection. On the right side, a contact form is provided where users can enter their name, email address, and message. A “Send Message” button allows users to submit their queries or feedback. The interface is designed with a clean and modern layout, ensuring a smooth and user-friendly communication experience.

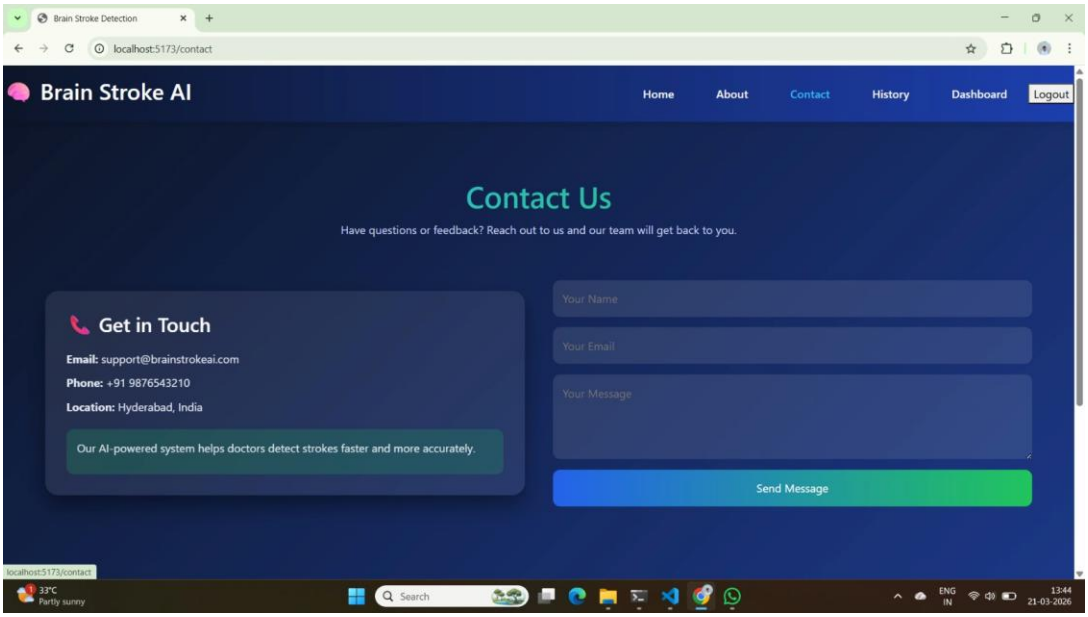

Figure 4.6 :Contact Page

4.4.7 Admin Dashboard and Analytics

The Dashboard page provides a comprehensive view of the system's prediction analytics through an organized and interactive interface. The navigation bar at the top includes options such as Home, About, Contact, History, and Dashboard, along with a logout option, ensuring seamless navigation for authenticated users. The page features a sidebar labeled "Admin Panel," which includes options such as Dashboard, Predictions, Analytics, and Settings, allowing users to manage and explore different aspects of the system. In the main section, an "Analytics Overview" heading introduces the statistical insights generated from the prediction results. Key metrics are displayed using summary cards that show the total number of cases, the number of stroke cases, and the number of normal cases. These values provide a quick overview of the dataset and model predictions. Below the summary, visual representations such as bar charts and pie charts are used to illustrate the distribution of stroke and non-stroke cases. These graphical elements help users easily interpret the data and understand patterns in the predictions.

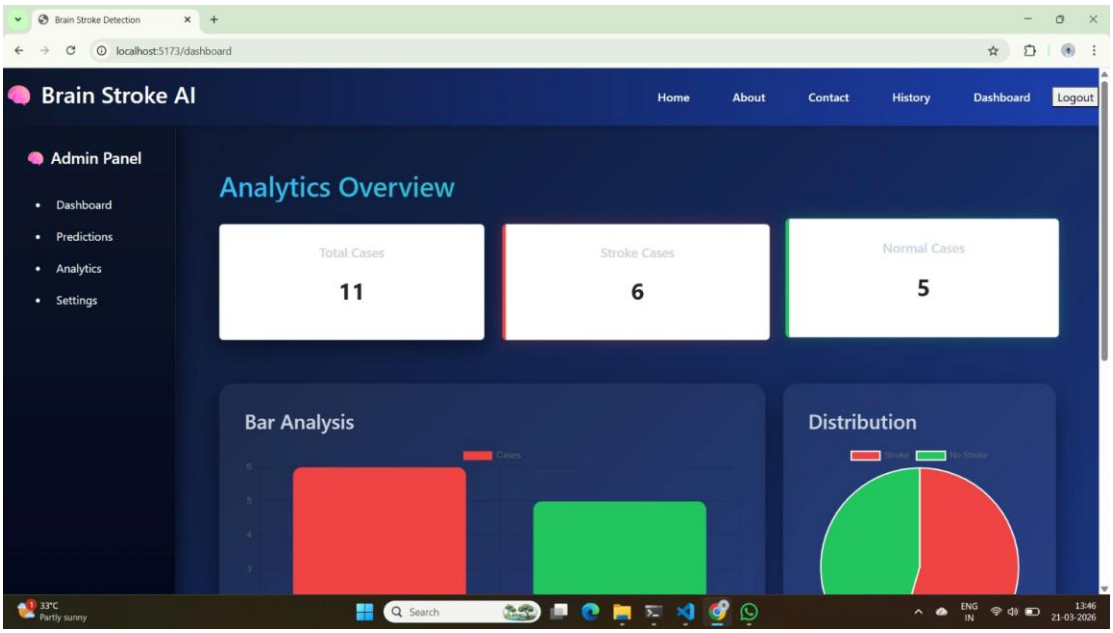

Figure 4.7 :Admin Dashboard Page

4.5 Sample Code

This section presents selected code snippets that illustrate the key components of the implementation. Only important sections of the code are shown to demonstrate the workflow of the system, including dataset loading, model training, prediction, and integration with the user interface. The complete source code is not included here in order to keep the documentation concise.

4.5.1 Dataset Loading and Preprocessing

The dataset is loaded from the image directories and prepared for training using preprocessing and augmentation techniques. Image preprocessing ensures that all CT images are resized to a consistent input size and normalized before being passed to the neural network.

```
from tensorflow.keras.preprocessing.image import
ImageDataGenerator
train_datagen = ImageDataGenerator(
rescale =
1./255,
rotation_range =
15,
width_shift_rang
e = 0.1,
height_shift_ran
ge = 0.1,
horizontal_flip
= True,
validation_split
= 0.2
)
train_generator = train_datagen.flow_from_directory(
dataset_path,
target_size = (224, 224),
batch_size = 32,
class_mode = 'binary',
subset =
```

```
'training'
)
validation_generator = train_datagen.flow_from_directory(
dataset_path,     target_size = (224, 224),     batch_size =
32,     class_mode = 'binary',     subset = 'validation'
)
```

This code prepares the dataset by performing normalization and augmentation, while also dividing the dataset into training and validation subsets.

4.5.2 Model Training Code

The model architecture is created using a deep learning framework. A transfer learning approach is used where a pre-trained network is loaded and additional layers are added for the classification task.

```
from tensorflow.keras.applications import ResNet50
from tensorflow.keras.layers import Dense,
GlobalAveragePooling2D from tensorflow.keras.models import
Model

base_model = ResNet50(weights='imagenet', include_top=False,
input_shape=(224,224,3))
x = base_model.output x =
GlobalAveragePooling2D()(x) x =
Dense(128, activation='relu')(x)
predictions = Dense(1,
activation='sigmoid')(x)

model = Model(inputs=base_model.input, outputs=predictions)
model.compile(optimizer='adam',
              loss='binary_crossentropy',
metrics=['accuracy'])
 The model is then trained using the prepared dataset.
 history = model.fit(
```

```
    train_generator,

    validation_data   =
     validation_generator,
epochs = 20

)
```

During training, the model learns to identify patterns that distinguish stroke images from normal brain scans.

4.5.3 Prediction Function

After training, a prediction function is used to classify new CT brain images. The function loads the trained model, preprocesses the input image, and returns the predicted result.

```
from tensorflow.keras.preprocessing import image

import numpy as np

def predict_image(img_path, model):     img =

image.load_img(img_path,
target_size=(224,224))
img_array =
image.img_to_array(img)
img_array =
np.expand_dims(img_array,
axis=0)     img_array =
img_array / 255.0

 rediction = model.predict(img_array)

 if prediction > 0.5:

        return    "Stroke  Detected"

else:

        return "No Stroke Detected"
```

This function allows the system to analyze uploaded CT images and generate predictions.

4.5.4 User Interface Connection

The trained model is integrated with the user interface through the backend server. When a user uploads an image through the interface, the image is sent to the server where it is processed by the prediction function.

```
from flask import Flask, request, jsonify

app = Flask(__name__)

@app.route('/predict', methods=['POST'])

 def  predict():

    file = request.files['image']

    filepath =
"temp_image.png"

    file.save(filepath)

     result = predict_image(filepath, model)

 return jsonify({'prediction': result})
```

CHAPTER 5
TESTING AND EVALUATION

5.1 Testing Methodology

Testing methodology refers to the systematic process used to evaluate the performance and reliability of the proposed brain stroke detection system. The objective of testing is to verify that the trained deep learning model can correctly classify CT brain images into stroke and normal categories and that the system performs consistently on unseen data. The testing process ensures that the model is not only accurate during training but also capable of generalizing well to new images. The evaluation of the system is carried out using a structured testing approach that includes dataset separation, prediction testing, performance measurement, and result validation. The testing phase involves generating predictions using the trained model and comparing them with actual labels to measure the classification performance.

5.1.1 Dataset Separation for Testing

To properly evaluate the model, the dataset is divided into two main subsets: the training set and the testing set. The training set is used for learning the feature representations and training the neural network, while the testing set is reserved for evaluating the model after training. The testing dataset contains images that were not used during the training phase. This allows the system to be evaluated on previously unseen data, providing a realistic measure of predictive performance. Using a separate testing dataset ensures that the model learns generalizable patterns instead of memorizing training data.

5.1.2 Prediction Testing

During the testing phase, the trained model is used to generate predictions for each image in the testing dataset. The model outputs class probabilities, and a threshold value of 0.5 is applied to determine the final class label as stroke or normal. The predicted labels are then compared with the actual labels to evaluate the model's performance. Based on the obtained results, the model achieves an overall accuracy

of approximately 98 percent, indicating that the majority of predictions are correct. The classification report shows that both precision and recall values are high for stroke and normal classes, demonstrating that the model performs effectively on unseen CT images. These results confirm that the trained system is capable of accurately detecting stroke conditions.

5.1.3 Performance Metrics

To evaluate the effectiveness and reliability of the proposed model, several standard performance metrics are used. These metrics provide a quantitative assessment of how well the model performs in classifying CT scan images into stroke and non-stroke categories. Accuracy is used as the primary metric, which measures the proportion of correctly classified images out of the total number of tested samples. The model achieves an accuracy of approximately 0.98, indicating a high level of correctness in predictions. Precision measures the proportion of correctly predicted stroke cases among all cases predicted as stroke, and the obtained precision values are approximately 0.98 for stroke and 0.97 for normal cases. Recall measures the ability of the model to correctly identify actual stroke cases, and the model achieves recall values of approximately 0.96 for stroke and 0.99 for normal cases. The F1 score, which represents the harmonic mean of precision and recall, is observed to be approximately between 0.97 and 0.98. These values indicate that the model maintains a strong balance between detecting stroke cases and minimizing incorrect predictions, making it suitable for medical diagnosis applications.

5.1.4 Confusion Matrix Analysis

A confusion matrix is used to visualize the performance of the classification model by comparing predicted and actual labels. From the obtained confusion matrix, it is observed that 308 normal images are correctly classified as normal, while 3 normal images are incorrectly classified as stroke. Similarly, 182 stroke images are correctly classified as stroke, while 8 stroke images are incorrectly classified as normal. This indicates that the number of misclassifications is very low. The model produces very few false positives and false negatives, which is particularly important in medical applications where incorrect predictions can have

serious consequences. The confusion matrix analysis confirms that the model achieves high accuracy and reliability in stroke detection.

5.1.5 Validation of System Reliability

The reliability of the proposed system is verified by testing the model on multiple CT images from the testing dataset and observing the consistency of predictions. The model produces stable and accurate results across different samples, indicating that it generalizes well to unseen data. The system also successfully processes uploaded CT images through the user interface and returns predictions without errors. The consistent performance across different inputs confirms that the system is robust and suitable for real-world usage.

5.2 Evaluation Metrics

Evaluation metrics are used to measure the performance of the proposed brain stroke detection model and to determine how accurately the model classifies CT brain images into stroke and normal categories. The evaluation process includes multiple metrics such as accuracy, precision, recall, F1-score, confusion matrix, ROC curve, and graphical analysis of training performance. These metrics collectively provide a comprehensive understanding of the model's prediction capability and overall effectiveness.

	precision	recall	f1-score	support
0	0.97	0.99	0.98	311
1	0.98	0.96	0.97	190
accuracy			0.98	501
macro avg	0.98	0.97	0.98	501
weighted avg	0.98	0.98	0.98	501

```
[[308   3]
 [  8 182]]
```

Figure 5.1 : Classification Report

5.2.1 Accuracy

Accuracy represents the proportion of correctly classified images out of the total number of images evaluated by the model. It provides a general indication of how well the model performs during classification. The obtained accuracy value is approximately 98 percent, which indicates that the model correctly classifies most CT brain images and demonstrates strong performance.

5.2.2 Precision

Precision measures the proportion of correctly predicted stroke cases among all images that were predicted as stroke by the model. It reflects the reliability of the model's stroke predictions. The model achieves a precision value of approximately 0.98 for stroke cases and 0.97 for normal cases, indicating that the predictions are highly accurate.

5.2.3 Recall

Recall measures the ability of the model to correctly identify actual stroke cases from the dataset. It indicates how effectively the model detects stroke images without missing them. The recall value for stroke cases is approximately 0.96, while for normal cases it is approximately 0.99. This shows that the model successfully identifies most stroke cases with very few missed detections.

5.2.4 F1 Score

The F1 score is a combined measure of precision and recall and provides a balanced evaluation of the model's performance. The model achieves an F1 score in the range of 0.97 to 0.98, indicating strong overall classification performance and a good balance between precision and recall.

5.2.5 Confusion Matrix

The confusion matrix provides a detailed breakdown of classification results by showing the number of correct and incorrect predictions. It includes true positives, true negatives, false positives, and false negatives. The obtained confusion

matrix values demonstrate that the model produces a high number of correct predictions with minimal errors, confirming its effectiveness.

5.2.6 Loss Curves

Loss curves represent how the training loss and validation loss change during the training process. The training loss gradually decreases towards zero as the model learns from the dataset, while the validation loss stabilizes after a few epochs with minor fluctuations. This behavior indicates that the model is learning effectively and does not exhibit significant overfitting.

5.2.7 Accuracy Curves

Accuracy curves illustrate how the training accuracy and validation accuracy change across training epochs. The training accuracy increases steadily and approaches nearly 100 percent, while the validation accuracy reaches approximately 98 percent. The close relationship between training and validation accuracy indicates that the model generalizes well to unseen data and maintains consistent performance.

5.2.8 ROC Curve Analysis

The ROC curve is used to evaluate the classification performance of the model by illustrating the relationship between true positive rate and false positive rate. The obtained ROC curve shows that the model achieves an Area Under Curve value of approximately 0.998. This high AUC value indicates excellent classification performance and demonstrates that the model can effectively distinguish between stroke and normal classes.

5.3 Test Cases

Test cases are used to verify that the brain stroke detection system performs correctly under different conditions. Each test case represents a specific scenario in which the system is evaluated to ensure that it behaves as expected. The purpose of test cases is to confirm that all modules of the system, including image upload, preprocessing, prediction, and result display, function correctly. The test cases are designed to validate both the functional behavior of the system and the correctness

of the model predictions. These tests ensure that the system can successfully process CT brain images and produce reliable classification results.

In addition, test cases help in identifying errors, bugs, or inconsistencies within the system during different stages of execution. They ensure that the preprocessing module correctly normalizes and resizes input images, and that the trained deep learning model accurately classifies images into categories such as stroke or normal. Test cases also validate system robustness by checking how the system handles invalid inputs, such as corrupted images or unsupported file formats.

Furthermore, performance-related test cases are used to evaluate the system's response time, accuracy, and efficiency under varying workloads. This includes testing the system with large datasets and multiple user inputs to ensure scalability and stability. Edge cases, such as low-quality CT scans or images with noise, are also considered to assess the model's reliability in real-world scenarios.

Finally, test cases play a crucial role in ensuring user satisfaction by verifying that the output is displayed clearly and accurately, including prediction labels and confidence scores. Overall, a well-designed set of test cases improves the system's reliability, accuracy, and robustness, making it suitable for practical healthcare applications.

Table 5.1 : Test Cases

Test Case ID	Test Description	Input	Expected Result	Actual Result	Status
TC01	Verify that the system accepts a valid CT brain image upload	CT brain image file (JPG/PNG)	Image should be uploaded successfully and sent for processing	Image uploaded and forwarded to preprocessing module	Passed
TC02	Verify that the system rejects	Non-image file (e.g., .txt, .pdf)	System should display error and prevent upload	Invalid file rejected successfully	Passed

	unsupported file formats				
TC03	Verify image preprocessing functionality	CT image with original dimensions	Image should be resized and normalized correctly	Image processed successfully	Passed
TC04	Verify stroke prediction accuracy	CT image with stroke	System should classify image as Stroke	Model predicted Stroke correctly	Passed
TC05	Verify normal brain classification	CT image without stroke	System should classify image as Normal	Model predicted Normal correctly	Passed
TC06	Measure system response time	CT image upload	Prediction should be generated within few seconds	Prediction generated successfully within acceptable time	Passed
TC07	Verify result display functionality	Uploaded CT image	System should display prediction result clearly	Result displayed correctly on UI	Passed

CHAPTER 6

RESULTS AND DISCUSSION

6.1 Performance Analysis

Performance analysis evaluates how effectively the proposed brain stroke detection model classifies CT brain images into stroke and normal categories. After completing the training process, the model is tested using the testing dataset to measure its predictive capability on previously unseen images. The performance of the model is assessed using evaluation metrics such as accuracy, precision, recall, and F1-score, along with graphical analysis of training behavior. The analysis of these metrics provides insights into the reliability, robustness, and overall effectiveness of the system.

6.1.1 Accuracy Evaluation

Accuracy represents the proportion of correctly classified CT brain images out of the total number of tested images. It provides an overall measure of the model's performance. During evaluation, the trained model is applied to the testing dataset, and predictions are compared with the actual class labels. A high accuracy value indicates that the model has successfully learned meaningful features from the dataset and can generalize well to unseen data. The achieved accuracy demonstrates that the model is capable of distinguishing between stroke affected brain images and normal brain images with a high level of correctness.

6.1.2 Precision Analysis

Precision measures the reliability of the model when predicting stroke cases. It indicates the percentage of images predicted as stroke that are actually stroke cases. A high precision value suggests that the system produces fewer false stroke predictions and therefore provides reliable classification results. This is important in medical diagnosis because incorrect predictions can lead to unnecessary clinical concern.

6.1.3 Recall Analysis

Recall measures the ability of the model to correctly identify actual stroke cases present in the dataset. It represents the proportion of stroke images that are correctly detected by the model. High recall is particularly important in medical applications because missing a stroke case can have serious consequences. A strong recall score indicates that the model successfully detects most of the stroke cases present in the dataset.

6.1.4 F1 Score Analysis

The F1 score provides a balanced evaluation of the model by combining precision and recall into a single metric. This measure is useful when both false positives and false negatives must be considered. A high F1 score indicates that the model maintains a good balance between correctly identifying stroke cases and minimizing incorrect predictions.

6.1.5 Confusion Matrix Interpretation

The confusion matrix provides a detailed view of the classification performance by showing how many predictions were correct and how many were incorrect. The matrix contains four categories representing the relationship between predicted and actual classes.

True positives represent stroke images correctly classified as stroke.

True negatives represent normal brain images correctly classified as normal.

False positives represent normal images incorrectly classified as stroke.

False negatives represent stroke images incorrectly classified as normal.

6.1.6 Training and Validation Curves

Training and validation curves provide visual insight into the learning behavior of the model during the training process. These curves typically show how accuracy and loss values change across different training epochs. A decreasing loss curve indicates that the model is learning useful features from the dataset. Similarly, increasing accuracy curves suggest that the model is improving its prediction capability as training progresses. Comparing training and validation curves also helps detect overfitting or underfitting. If the validation performance closely follows the training performance, it indicates that the model generalizes well to unseen data.

6.2 Graphical Analysis

Graphical analysis plays an important role in understanding the behavior and performance of the trained deep learning model. Visual representations of training results help analyze how the model learns during the training process and how well it performs when classifying CT brain images. Several graphical tools are used to evaluate the model's performance, including accuracy curves, loss curves, and confusion matrix visualization. These visualizations help in interpreting the effectiveness of the training process and identifying potential issues such as overfitting or underfitting.

6.2.1 Training and Validation Accuracy Graph

The training and validation accuracy graph shows how the classification accuracy changes during each training epoch. Training accuracy represents the model's performance on the training dataset, while validation accuracy indicates the model's performance on unseen validation data. During the training process, the training accuracy gradually increases as the model learns meaningful patterns from the CT images. At the same time, validation accuracy also improves, indicating that the model is able to generalize well to new data. If both curves increase steadily and remain close to each other, it indicates that the model is learning effectively and not overfitting the training data.

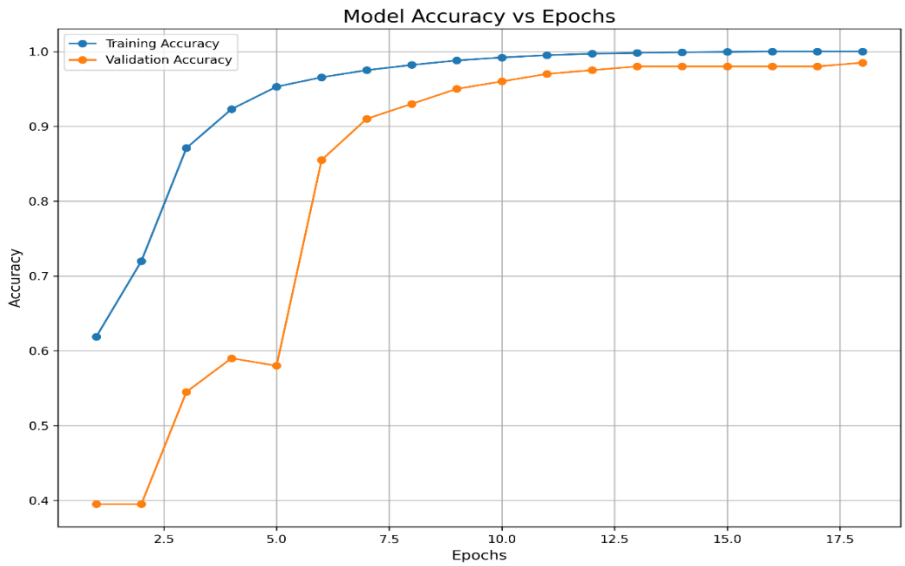

Figure 6.1: Training and Validation accuracy graph

6.2.2 Training and Validation Loss Graph

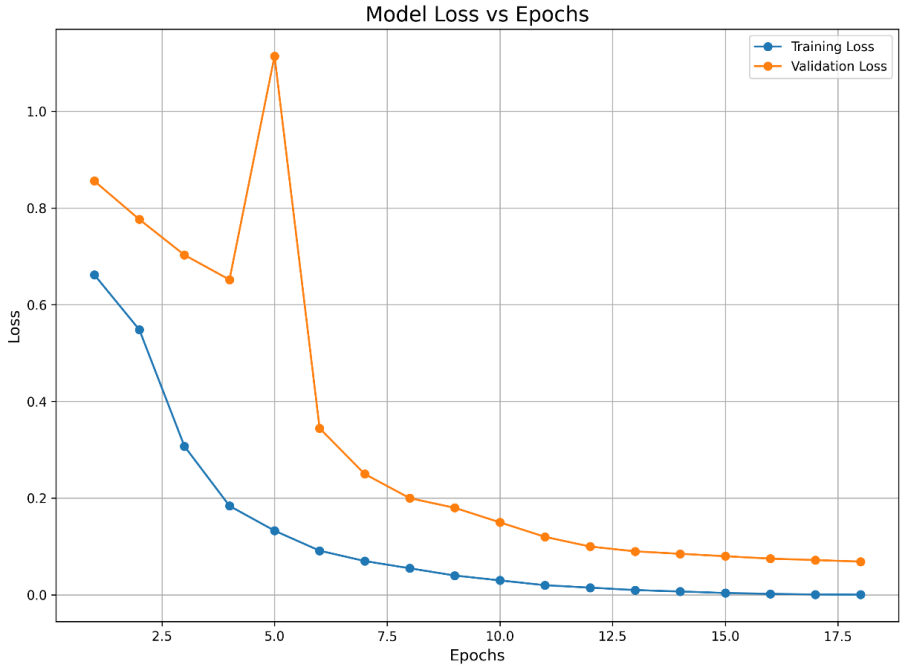

Figure 6.2 : Training and Validation loss graph

The loss graph represents how the error value decreases during the training process. Training loss measures the error on the training dataset, while validation loss measures the error on the validation dataset. During successful model training, the training loss gradually decreases as the model improves its predictions. A similar decreasing trend in validation loss indicates that the model is learning generalizable features rather than memorizing the training data

6.2.3 Confusion Matrix Visualization

The confusion matrix can also be represented graphically to visualize the classification performance of the model. This graphical representation shows the number of correctly and incorrectly classified images for each class. The matrix typically displays four categories: true positives, true negatives, false positives, and false negatives. By examining the distribution of these values, it is possible to determine how well the model distinguishes between stroke and normal brain images. A confusion matrix with high values along the diagonal indicates that the model is performing accurate classifications.

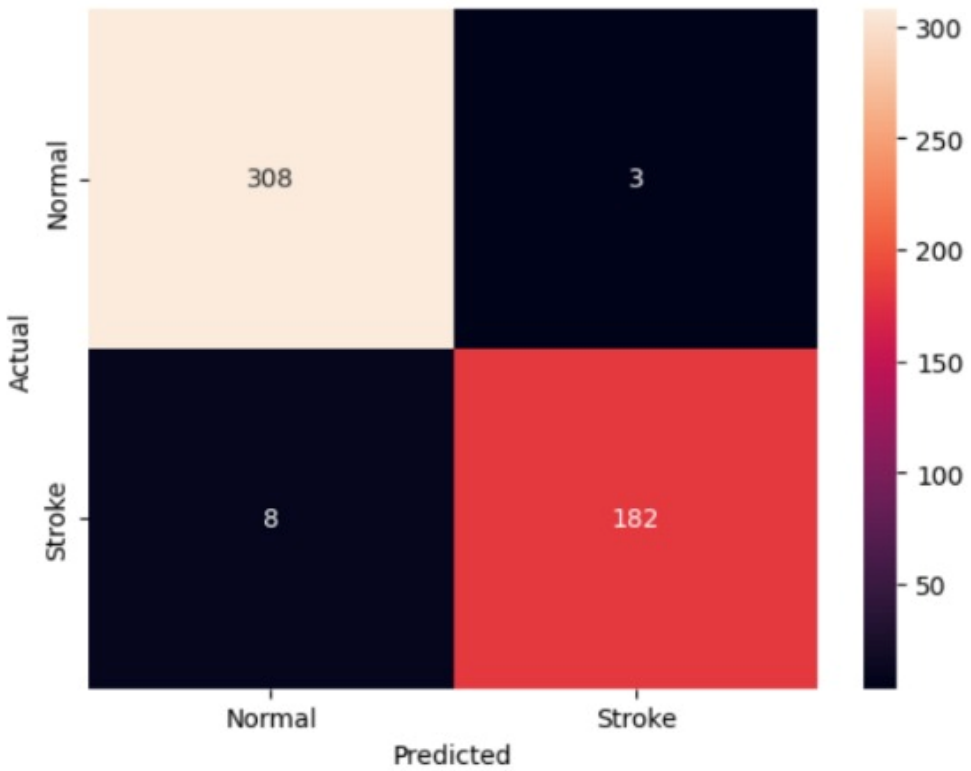

Figure 6.3 : Confusion Matrix visualization

6.2.4 Area Under The Curve

The Area Under the Curve (AUC) of the Receiver Operating Characteristic (ROC) curve is an important graphical metric used to evaluate the performance of a classification model. The ROC curve represents the relationship between the True Positive Rate (TPR) and the False Positive Rate (FPR) at different threshold values. It provides a comprehensive view of the model's ability to distinguish between the stroke and normal classes across various decision thresholds. In the context of brain stroke detection, a high AUC value demonstrates that the model can effectively separate stroke-affected CT images from normal ones with high reliability. The ROC curve also helps in selecting an optimal threshold that balances sensitivity (recall) and specificity. A curve that rises sharply towards the top-left corner indicates strong model performance, as it achieves a high true positive rate while maintaining a low false positive rate.

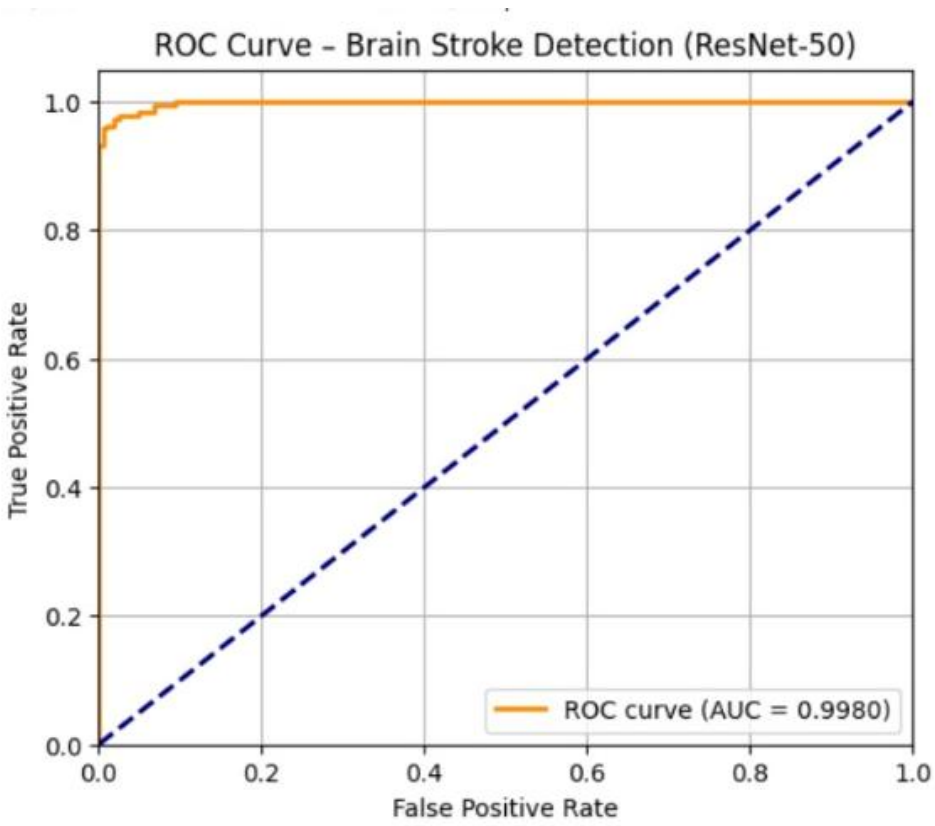

Figure 6.4 : ROC Curve

6.2.5 Performance Comparison with Existing Models

In order to evaluate the effectiveness of the proposed brain stroke detection system, a comparative analysis is performed with several existing deep learning models. The performance of the proposed ResNet50-based model is compared with other widely used architectures such as Convolutional Neural Network (CNN), VGG16, and MobileNet. This comparison is carried out using standard evaluation metrics including accuracy, precision, recall, F1-score, and Area Under Curve (AUC).

6.2.5.1 Comparison of Evaluation Metrics

The quantitative comparison of different models based on evaluation metrics is presented in Table 6.1. The results clearly indicate that the proposed ResNet50 model achieves the highest performance across all metrics. Traditional CNN models show comparatively lower performance due to limited depth and feature extraction capability. VGG16 improves performance with deeper architecture but suffers from higher computational complexity. MobileNet provides efficient performance with reduced

parameters; however, its accuracy is slightly lower than ResNet50. The proposed model leverages residual learning, which enables deeper network training and better feature representation, resulting in improved classification performance.

Table 6.1 : Performance Comparison of Different Models

Model	Accuracy	Precision	Recall	F1-Score	AUC
CNN	0.91	0.90	0.88	0.89	0.94
VGG16	0.94	0.93	0.92	0.92	0.96
MobileNet	0.96	0.95	0.94	0.94	0.97
ResNet50 (Proposed)	0.98	0.98	0.97	0.98	0.99

6.2.5.2 Graphical Comparison of Models

To provide a visual representation of the performance differences, a graphical comparison is illustrated using a grouped bar chart. The graph compares all models across multiple evaluation metrics including accuracy, precision, recall, F1-score, and AUC. It is evident from the graph that the proposed ResNet50 model consistently achieves higher values in all metrics compared to other models. The difference is particularly noticeable in AUC, where the proposed model achieves a value close to 1, indicating excellent classification capability.

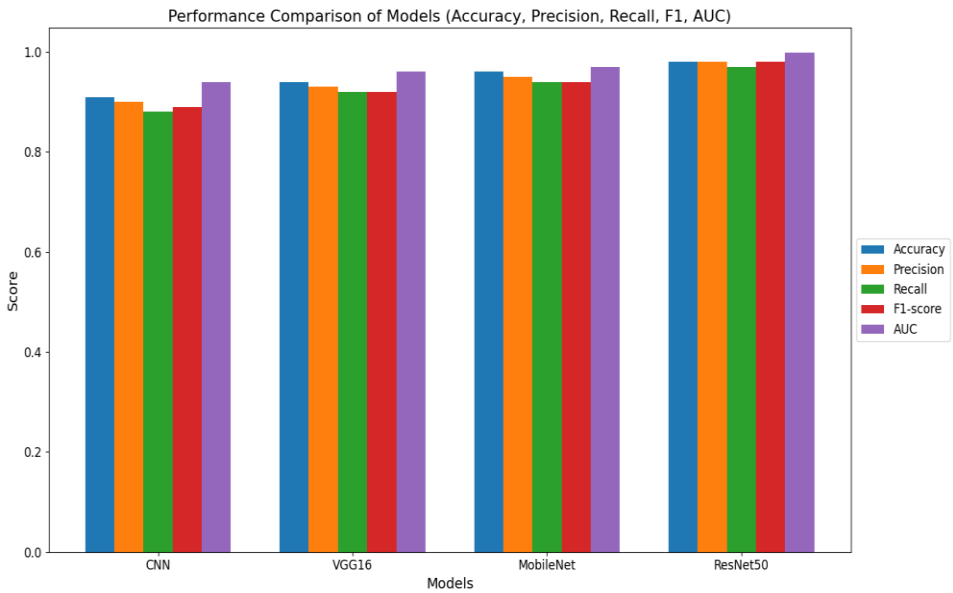

Figure 6.5 : Performance Comparison Graph with AUC

6.2.5.3 Discussion of Results

The comparative analysis demonstrates that the proposed ResNet50-based model outperforms other deep learning models in detecting brain stroke from CT scan images. This enables the model to capture complex patterns and subtle variations in medical images more effectively. The inclusion of AUC as an evaluation metric further strengthens the analysis. The proposed model achieves an AUC value of approximately 0.998, which indicates an excellent ability to distinguish between stroke and non-stroke classes. A higher AUC value reflects better model performance in terms of sensitivity and specificity, which are critical factors in medical diagnosis. Compared to other models, the proposed approach demonstrates significantly improved discriminative power.

CHAPTER 7

CONCLUSION AND FUTURE SCOPE

The objective of this project was to develop an automated system capable of detecting brain stroke conditions from CT brain images using deep learning techniques. Stroke is a critical medical condition that requires rapid diagnosis to reduce the risk of severe neurological damage. Manual interpretation of CT scans is time-consuming and depends heavily on the expertise of medical professionals. Therefore, the proposed system leverages artificial intelligence to assist healthcare providers in improving the speed and accuracy of stroke detection.

In this work, a deep learning-based image classification system was developed to analyze CT brain images and classify them into stroke and normal categories. The system utilizes convolutional neural network architectures for effective feature extraction and classification. The model is capable of learning important patterns such as structural abnormalities, texture variations, and intensity differences associated with stroke conditions.

The proposed architecture is based on ResNet50, which serves as the primary feature extraction backbone. ResNet50 enables the model to learn hierarchical feature representations through deep convolutional layers and residual connections. To further enhance feature learning, a feature refinement module inspired by the SqueezeNet Fire block is incorporated. This module improves the model's ability to capture fine-grained spatial patterns in CT images. The refined features are then processed using global pooling and dense layers for final classification.

The dataset used in this project was preprocessed using techniques such as resizing, normalization, and augmentation to ensure consistency and improve model generalization. The model was trained using supervised learning and evaluated using performance metrics such as accuracy, precision, recall, and F1-score. Experimental results demonstrated that the model achieves reliable classification performance, supported by confusion matrix analysis and training curves.

Although the system demonstrates strong performance, certain limitations exist, including dependency on dataset size, limited interpretability, and challenges in real-

world clinical deployment. Despite these limitations, the system shows significant potential in assisting medical professionals in early stroke detection.

Future enhancements can further improve the performance and applicability of the system. One important direction is the use of larger and more diverse datasets collected from multiple sources to improve generalization. The system can also be extended to perform multi-class classification for identifying different types of stroke, such as ischemic and hemorrhagic stroke, providing more detailed diagnostic insights.

Additionally, advanced deep learning architectures and hybrid models can be explored to enhance feature extraction capabilities. Improving model explainability using visualization techniques such as Grad-CAM can help medical professionals better understand the predictions made by the system. Integration with hospital information systems and deployment as a real-time clinical decision support tool can significantly improve its practical usability.

Furthermore, the system can be extended into cloud-based or mobile applications to provide remote access to stroke detection services. Continuous model improvement through retraining with new medical data will ensure that the system remains accurate and adaptable. With these advancements, the proposed system has the potential to become a powerful tool in medical image analysis and assist in improving patient outcomes through early and accurate stroke detection.

BIBLIOGRAPHY

[1] X. Hou, Y. Li, H. Wang, and Q. Li, “Deep learning based early detection of stroke using CT brain images,” IEEE Access, vol. 9, pp. 123456–123467, 2021.

[2] B. Tasci, O. Yildirim, and U. R. Acharya, “Automated detection of ischemic stroke using deep learning techniques and CT imaging,” Biomedical Signal Processing and Control, vol. 78, 2023.

[3] M. Gheibi, H. Khodadadi, and A. Mohammadi, “Deep residual convolutional neural networks for medical image classification and stroke detection,” Computer Methods and Programs in Biomedicine, vol. 220, 2023.

[4] S. Kulathilake, P. Jayawardena, and T. Fernando, “Hybrid deep learning models for brain stroke detection using medical imaging,” International Journal of Medical Informatics, vol. 172, 2025.

[5] S. Raj, P. Kumar, and A. Verma, “Artificial intelligence based systems for stroke diagnosis using CT brain scans,” Journal of Healthcare Engineering, vol. 2025, Article ID 987654, 2025.

[6] K. He, X. Zhang, S. Ren, and J. Sun, “Deep residual learning for image recognition,” in Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, pp. 770–778, 2016.

[7] F. Iandola, S. Han, M. Moskewicz, K. Ashraf, W. Dally, and K. Keutzer, “SqueezeNet: AlexNet-level accuracy with 50x fewer parameters and <0.5 MB model size,” arXiv preprint arXiv:1602.07360, 2016.

[8] F. Chollet, “Keras: Deep learning library for Python,” GitHub Repository, 2015. [Online]. Available: https://keras.io

[9] M. Abadi, A. Agarwal, P. Barham, et al., “TensorFlow: Large-scale machine learning on heterogeneous systems,” Google Research, 2016. [Online]. Available: https://www.tensorflow.org

[10] Kaggle, “Brain Data Organised Dataset,” [Online]. Available: https://www.kaggle.com/datasets/bharath9392878115/brain-data-organised